

ENGINEERING BLUEPRINT · VERSION 1.0

QUIP

Master Future

Vision

The single source of truth for where Quip is going: a lightweight, AI-native desktop companion built on compact understanding, real computer control, verified execution, and honest failure. Thirty-two chapters covering the capability model, the task lifecycle, memory and learning, model routing, the six companions, security, testing, and the roadmap from prototype to product.

Dhruv Sharma & Carren Mathew

Quip Engineering · September 2026

Table of Contents

1. Executive Summary: What Quip Is and Is Not	4
1. Executive Summary: What Quip Is and Is Not	4
2. Core Philosophy: More Capability Per Byte, Token, Second	7
2. Core Philosophy: More Capability Per Byte, Token, Second	7
3. The Long-Term Vision	9
3. The Long-Term Vision	10
4. System Architecture: Current and Target	12
4. System Architecture: Current and Target	12
5. The Capability Model	15
5. The Capability Model	15
6. Environment and Device Understanding	19
6. Environment and Device Understanding	19
7. Screen Understanding and Computer Control	22
7. Screen Understanding and Computer Control	22
8. Browser Control, Web Reading and Agent Reach	25
8. Browser Control, Web Reading and Agent Reach	25
9. Filesystem, Applications and OS Control	27
9. Filesystem, Applications and OS Control	27
10. The Task Lifecycle	30
10. The Task Lifecycle	30
11. Understanding Engine	33
11. Understanding Engine	33
12. Planning, Permissions and Execution	35
12. Planning, Permissions and Execution	35
13. Observation, Verification, Recovery and Cancellation	38
13. Observation, Verification, Recovery and Cancellation	38
14. Memory: Compact Knowledge, Not Bloated History	41
14. Memory: Compact Knowledge, Not Bloated History	41
15. Workflow Learning and Procedural Knowledge	43
15. Workflow Learning and Procedural Knowledge	43

16. Model Routing and Provider Abstraction	45
16. Model Routing and Provider Abstraction	45
17. Voice and Multimodal Interaction	48
17. Voice and Multimodal Interaction	48
18. The Six Companions	50
18. The Six Companions	50
19. Desktop Presence, Chatbox and Themes	52
19. Desktop Presence, Chatbox and Themes	52
20. Settings, Permissions UX and Proactive Behavior	55
20. Settings, Permissions UX and Proactive Behavior	55
21. Performance and Resource Efficiency	57
21. Performance and Resource Efficiency	57
22. Caching and Indexing Strategy	60
22. Caching and Indexing Strategy	60
23. Security, Privacy and Trust Boundaries	62
23. Security, Privacy and Trust Boundaries	62
24. Reliability Engineering and Lifecycle	64
24. Reliability Engineering and Lifecycle	64
25. Current Codebase Audit	66
25. Current Codebase Audit	66
26. Skales Integration Strategy	69
26. Skales Integration Strategy	69
27. Testing Strategy and Observability	71
27. Testing Strategy and Observability	71
28. Packaging, Updates and Migration	73
28. Packaging, Updates and Migration	73
29. Future Capabilities	75
29. Future Capabilities	75
30. Worked Examples: Task Walkthroughs	78
30. Worked Examples: Task Walkthroughs	78
31. Roadmap: From Prototype to Vision	80
31. Roadmap: From Prototype to Vision	80

32. Appendices	82
32. Appendices	82

1. Executive Summary: What Quip Is and Is Not

1. Executive Summary: What Quip Is and Is Not

Quip is an AI life companion that lives on the user's desktop as a persistent, visible character. It is not a chat window that opens and closes; it is a resident presence that observes context, understands requests, executes real work on the computer, verifies what actually happened, and reports honestly. The product thesis is simple to state and hard to build: the companion is the interface, the computer is the workspace, and understanding is the engine. Everything in this document follows from taking that thesis seriously as an engineering discipline rather than as marketing language.

The current codebase, audited in Chapter 25, already contains a substantial portion of this system: a modular understanding pipeline, a separated planning and execution stack, a six-provider model failover chain, real voice output through a three-engine chain, screen vision, browser automation, a device knowledge layer with diff-only rescans, persistent memory with importance scoring, six distinct companions, ten complete themes, and a 425-test suite. It also contains contradictions, dead subsystems, startup-order defects, and unbounded storage paths that this document identifies precisely and schedules for removal or repair. Honesty about the current state is a design principle: nothing in this document claims a capability that the code cannot demonstrate.

Quip is not a general agent platform, not a cloud service, not a wrapper around a single model provider, and not a collection of hardcoded phrase triggers. It is explicitly not designed to accumulate enormous local databases or to send entire conversation histories to a model on every turn. The governing objective for the next phase of development can be compressed into three sentences: more capability per byte of storage, more capability per token of model context, and more capability per second of user waiting. Each of these is expanded into concrete architectural mechanisms in the chapters that follow.

3× Efficiency Thesis

More capability per BYTE of storage · per TOKEN of context · per
SECOND of user waiting

This document serves three audiences at once. For the project owner, it is the single source of truth for where Quip is going, so that individual feature requests can be evaluated against a coherent direction instead of accumulating as isolated layers. For an engineer joining the project, it is a self-contained explanation of how the pieces fit: the lifecycle in Chapter 10, the capability model in Chapter 5, the audit in Chapter 25, and the roadmap in Chapter 31 can be read in that order as an onboarding path. For the autonomous build process itself, it is the blueprint from which the 400-task engineering backlog was generated and against which every change is verified.

How to read this document

Chapters 1 through 4 establish philosophy and architecture. Chapters 5 through 9 define the capability model and each control domain: environment, screen, browser, and filesystem. Chapters 10 through 15 walk the task lifecycle end to end, from intent understanding through execution, verification, recovery, memory, and learning. Chapters 16 through 20 cover intelligence routing, voice, the companion system, and the product experience. Chapters 21 through 24 cover the operational disciplines: performance, caching, security, and reliability. Chapter 25 contains the verified codebase audit; Chapters 26 through 28 cover integrations, testing, and packaging; Chapters 29 through 31 present future capabilities, worked examples, and the roadmap. The appendices carry the capability map, the IPC registry digest, and a glossary.

The one-paragraph current state

As of the audit baseline, Quip builds cleanly, passes 425 tests, boots to a companion sprite with four screen modes, streams chat through a six-provider failover chain, speaks replies through a three-engine voice chain, executes direct and agent-shaped tasks through a single live pipeline with an execution log, and keeps its mascot alive unless the user quits from Settings. Its defects are specific rather than diffuse: a first-run startup order that hides the scan behind a blocking bootstrap, two unbounded JSON stores, a deterministic execution path without deadlines, a fabricated weekly digest, a permission system with no visible switch, and a layer of theme-token leaks and dead code. None of these is a mystery; each has evidence, an owner, a fix strategy, and a test, which is what makes this document an engineering plan rather than a wish list.

What the reader should hold onto

If everything else is forgotten, hold onto three sentences. The companion is the interface, so presence and honesty are product features, not polish. The computer is the workspace, so every capability is a primitive with a contract, a risk class, and a verification. Understanding is the engine, so context is assembled deliberately and determinism is preferred wherever knowledge suffices. Every chapter that follows is one of those sentences turned into modules, budgets, and tests.

Question this document answers	Chapter
What is Quip and what must it never become?	1-3
How is the system architected today and tomorrow?	4
What can it do, and through what primitives?	5-9
How does a request become verified work?	10-13
How does it remember and learn without bloating?	14-15
How is intelligence routed and paid for?	16
What does the user actually see and feel?	17-20

Question this document answers	Chapter
How fast, safe, and reliable is it?	21-24
What is broken today, precisely?	25
What happens next, in what order?	29-31

Table 1-1 · Question-to-chapter index

Three readers, one document

The owner reads this document to keep the product coherent: every new idea lands against a chapter, and anything that cannot find a chapter gets priced before it gets built. The engineer reads it as an onboarding path, Chapter 10's lifecycle, Chapter 5's capability model, Chapter 25's audit, Chapter 31's roadmap, and should be able to trace any line of code back to one of those four. The build process itself reads it as the source of the backlog and the arbiter of disputes: when two implementations conflict, the one consistent with this document stays, and the reason is recorded in the architecture-decisions ledger rather than argued from seniority. A document that cannot serve all three readers becomes shelfware; this one is structured so each has a spine through it.

How honesty is enforced in this document

Three conventions keep the document truthful. Present tense means the audit verified it in code at the stated commit; future tense or “planned” means it does not exist yet; and anything the sandbox could not execute is marked unverified with the environment it needs. Findings carry file-and-line evidence or they are not findings. Where the document states a budget, a test or a measurement plan exists for it, and where a chapter promises an artifact, the artifact's location is named in Appendix D. The result reads drier than a pitch deck and considerably more useful than one, which is the intent.

A vision document earns its keep when the next engineer can act on it without the conversation that produced it.

2. Core Philosophy: More Capability Per Byte, Token, Second

2. Core Philosophy: More Capability Per Byte, Token, Second

Every architectural decision in Quip is evaluated against three ratios, and each ratio has a concrete engineering meaning. Capability per byte means the system prefers compact, derived understanding over raw capture: a device index of a few hundred kilobytes that answers thousands of questions is worth more than gigabytes of screenshots; a learned procedure of ten steps is worth more than a thousand stored transcripts. Capability per token means the context sent to any model is assembled deliberately from indexes, summaries, and relevant slices rather than dumped wholesale; a task that needs the device index should not pay for the conversation history, and a task that needs history should receive its compressed gist, not its full text. Capability per second means deterministic execution is preferred whenever knowledge is sufficient: opening a known application must never wait on a model round trip.

These ratios translate into five standing rules. First, deterministic before model: if a fast path can resolve the request from the device index, learned procedures, or cached understanding, it runs without any model call, and the model is the fallback for genuine ambiguity rather than the default translator. Second, index before rescan: stable environment facts are cached with hashes and time-to-live, and only diffs are recomputed, because rescanning the whole machine to answer one question is waste. Third, summarize before store: anything persisted must justify its bytes by the probability it will change a future decision; raw logs live in bounded rings, not in permanent storage. Fourth, verify before claim: a task is complete only when observed screen or system state matches the plan's expectation contract, and failure is reported with its real cause, never masked. Fifth, compose rather than special-case: new abilities come from new combinations of existing primitives, not from new if-else branches keyed on phrases.

Anti-goals

- **No blind accumulation.** Storing every message, screenshot, or scan result “just in case” is forbidden; every persistent artifact has a cap, a decay rule, or a ring bound.
- **No phrase hardcoding.** The intent parser may contain hint tables, but behavior must trace to structured understanding and capability discovery, never to a lookup of exact sentences.
- **No silent failure.** Empty catch blocks that swallow an error and pretend success violate the trust contract; every failure path produces an honest, specific reason surfaced to the user.
- **No second engine.** When an implementation conflicts with the target architecture, it is refactored or deleted; running two parallel systems (for example, two planning pipelines) is a defect, not a safety net.

- **No model dependency for determinism.** If removing every API key still leaves a task executable from local knowledge, the architecture is correct; if not, the fast-path layer is incomplete.

The philosophy also defines the relationship between speed and correctness. Quip eliminates unnecessary waiting but never trades correctness for speed. A fast path is only as trusted as its verification: even a deterministic launch is confirmed by observing the process or window state, and if confirmation fails, the system falls back to deeper reasoning and reports what it found. In practice this ordering, deterministic attempt, verified outcome, model fallback on ambiguity, honest failure on exhaustion, gives the user both properties at once: most common actions complete in well under a second, and the hard cases still receive full intelligence.

The efficiency theses as testable rules

A philosophy that cannot fail a code review is decoration. Each thesis therefore lands as a rule with a failure condition. Capability per byte fails when a store exceeds its declared bound or when raw captures are persisted where derived summaries suffice. Capability per token fails when a prompt contains content that did not plausibly change the answer, verifiable by sampling prompts and asking that question of each block. Capability per second fails when a deterministic task waits on a model call that an index or procedure could have answered. The backlog's verification tasks implement exactly these checks: store-size assertions, prompt-audit sampling, and fast-path counters in the execution log.

Thesis	Rule	Failure signal	Verification artifact
Per byte	Every store has a bound and a decay or ring rule	store size assertion fails	test suite + diagnostics snapshot
Per token	Prompt blocks must earn their place by relevance	prompt audit shows dead weight	sampled prompt audits
Per second	Deterministic answers never cross the model boundary	fast-path counter regresses	execution log fast-path ratio
Trust	Success is claimed only from observed state	a claim without observation	verifier contract tests
Composition	New abilities extend registries, not if-else ladders	parser gains behavior keyed on phrases	architecture review checklist

Table 2-1 · Philosophy as enforceable engineering rules

Why the anti-goals are load-bearing

Each anti-goal encodes a scar. Blind accumulation produced the two unbounded stores the audit found, and it is the classic path from clever prototype to ten-gigabyte appdata folder. Phrase hardcoding is how chatbots die: every new phrasing becomes a new branch until nobody can say what the system will do with a sentence it has not seen. Silent failure is the single fastest trust destroyer in a companion product, because the user cannot distinguish a broken promise from an unkept one. The second-engine anti-goal

comes from direct evidence: the legacy planning trio survived precisely because a parallel implementation was kept “just in case,” and it shipped dead weight on every boot. Model dependency for determinism is the quiet one: products that let the model mediate every action become slow and expensive in ways no profiling can fix, because the cost is architectural.

Worked arithmetic: a day of tokens

The efficiency thesis becomes concrete with arithmetic. Suppose a heavy day: fifty deterministic tasks (opens, finds, toggles), ten chat exchanges, three agent-shaped plans, one research task with two web reads. Under the architecture, the fifty deterministic tasks cost zero tokens because the index and referents serve them; the ten exchanges cost their compact prompts, identity plus filtered memory plus gist, on the order of a thousand tokens each rather than ten thousand; the three plans spend their model attention only at decomposition and ambiguity points; the research task pays for compression of two reads. The day lands in the low tens of thousands of tokens where an undisciplined design that ships full history and full environment to every call would spend hundreds of thousands. The ratio is the product: same capability, one order of magnitude fewer tokens, which is what “more capability per token” means when it is not a slogan.

Worked arithmetic: a year of bytes

Storage arithmetic is similarly decisive. Memory at its cap is a few hundred kilobytes; the bounded knowledge graph and timeline land in the low megabytes worst case; the execution log's ring of 250 trails and the connection journal's ring of 80 are hundreds of kilobytes; the screenshot ring of eight is a few megabytes worst case; procedures are kilobytes each, and a hundred of them is still under a megabyte. The whole understanding footprint lives comfortably under fifty megabytes with the chat archive under the user's own control. A design that archived every transcript and screenshot instead would cross a gigabyte in months and begin swapping trust for storage, which is the trade this architecture refuses.

3. The Long-Term Vision

3. The Long-Term Vision

The long-term Quip is a lightweight, efficient, AI-native desktop companion that understands intent and performs extremely broad computer work while using as little storage, bandwidth, memory, and model usage as reasonably possible. It maintains a compact, continuously refreshed understanding of the user's environment: which applications are installed, which browser is default, which accounts are signed in, which projects and files matter, which workflows the user repeats. It reasons about objectives rather than commands, so the same machinery that handles “open VS Code” also handles “open my Quip project, find why the build is failing, fix it, run the tests, and tell me what happened.” It operates the computer through the same primitives a user does: applications, windows, mouse, keyboard, clipboard, files, and browsers, subject to permissions and verification.

The vision's resource discipline is explicit. The entire persistent footprint of understanding, memory, procedures, indexes, caches, excluding conversation logs the user can clear, targets a few tens of megabytes on disk and a working set that stays modest at idle. Network usage concentrates in model calls that earn their tokens: vision calls only when the screen genuinely must be seen, completion calls only when deterministic knowledge is insufficient, and each call's context assembled from compact summaries and relevant index slices. Startup targets seconds, not tens of seconds, because the device index is cached and diffed rather than rebuilt. The companion is present the moment the machine is ready for it, because a presence that makes you wait is a presence you close.

Resource	Budget (target)	Mechanism that enforces it
Persistent understanding store	< 50 MB total	Caps and decay on memory, ring buffers for logs, diff-based device index
Startup to visible companion	< 3 s warm, < 15 s first run	Cached device profile, window before bootstrap completes, background scan
Idle CPU	near-zero between events	Event-driven loops, no polling render cycles, debounced persistence
Model tokens per simple task	0 tokens	Deterministic fast paths from device index and learned procedures
Model tokens per complex task	compact context only	Relevance-filtered prompt assembly, summaries instead of raw history
Vision calls	only when observation requires sight	Verification uses cheap state checks first; vision is the escalation, not the default

Table 3-1 · Resource budgets and their enforcing mechanisms

Capability growth under this vision is composable rather than voluminous. When a new application, website, or file type appears, Quip does not grow a new subsystem; it extends indexes and the capability registry entries that already exist. When a workflow is performed successfully, its plan is compacted into a reusable procedure keyed by goal shape, so the second execution is faster and cheaper than the first.

The system is designed so that becoming more capable does not mean becoming heavier, which is the property that separates a companion that lives on your machine from an agent that consumes it.

A day with the long-term Quip

The vision is easiest to grasp as a day. Morning: the companion is already on screen from yesterday, position and mode remembered; the machine booted, the diff-only index refresh took moments in the background, and nothing asked for attention. A spoken sentence, “open my project and catch me up,” opens the workspace and summarizes overnight changes from its own execution log and the file index, not from a fresh interrogation of every tool. Midday: a compound request, research, draft, file, verify, runs as a supervised plan with checkpoints, and the user watches progress through the companion's states rather than a spinner. Evening: a repeated chore triggers a learned procedure that replays in seconds with verification, and the weekly digest, built from real data, shows the week's task trails. At no point did the user manage storage, keys, or modes unless they chose to; the day's model spend is a few cents of tokens at genuine decision points.

Why lightweight is a feature, not a constraint

A companion lives on someone's personal machine, which means it inherits the machine's politics: disk space, memory pressure, battery, and attention. Heavy software gets evicted. Lightness is therefore not an engineering preference but a survival property, and it compounds with trust: a system that stays small, stays quiet, and tells the truth earns the right to do more. This is also why the architecture refuses storage-heavy shortcuts like recording everything for later indexing, and token-heavy shortcuts like sending the whole world to a big model. The hard path, compact understanding maintained by diffs and procedures, is the only path that ends with the companion still welcome on the machine a year later.

Dimension	Prototype today	Vision state
First-run to companion	tens of seconds, scan invisible	window first, scan visible and honest
Persistent understanding	two stores unbounded	everything bounded and decayed
Repeat task cost	re-planned each time	learned procedure replays verified
Deterministic share of tasks	high but unmeasured	measured, budgeted, rising
Permission governance	engine only	visible modes with remembered grants
Failure story	mostly honest, gaps exist	every failure named, evidenced, testable

Table 3-2 · Prototype to vision, dimension by dimension

Non-functional requirements as contracts

NFR	Contract	Verified by
Responsiveness	UI state changes render within one frame; tasks report within their deadline	interaction tests; deadline stack
Presence	companion visible after boot within budget; persists across close and reboot	lifecycle matrix rows 1-4
Honesty	no success without observation; no error without reason	verifier contract tests; log audits
Frugality	all stores bounded; tokens budgeted per call	store-size assertions; token ledger
Sovereignty	all personal data local; egress legible; keys never leave main	security review checklist
Recoverability	any crash, kill, or power loss leaves a bootable state	clean-start and crash tests

Table 3-3 · Non-functional requirements as testable contracts

Scope boundaries of the vision

The vision's boundary is as deliberate as its content. Quip does not aim to be an operating system, a browser, or a general agent platform; it aims to be the companion layer that operates those things through their public surfaces. It does not replace the user's judgment on consequential actions; it asks, remembers the answer, and acts within it. It does not store the user's life; it stores the compact understanding that makes help possible. And it does not grow by accretion; this document is the filter, and features that would compromise the lightweight contract are declined here rather than debugged later.

4. System Architecture: Current and Target

4. System Architecture: Current and Target

The as-built architecture, verified by direct code reading during the audit, is organized as an Electron main process containing the intelligence and control systems, a React renderer containing presence and interaction, and a typed IPC bridge between them. The live execution spine is: TASK_EXECUTE IPC enters a brain hub that produces a structured understanding, routes direct plans to a verified action engine with recovery and an execution log, and routes agent-shaped requests through an orchestrator, an agent loop, and a tool registry of more than forty capabilities including desktop control, screen vision, browser automation, file operations, and web reading. The chat spine streams from a six-provider model router with a circuit breaker and per-provider model fallback. Presence is a single window factory rendering four modes, companion, panel, full, and fullscreen, with the close button semantically bound to hide-not-quit and the only quit path in Settings.

Process and data flow

Layer	Modules (representative)	Responsibility
Entry	main.ts	Lifecycle, window factory, IPC registration, prompt assembly, health probes
Understanding	brain/hub, understanding, references, intent-parser-v2	Messages to structured understanding objects; pronoun and “again” resolution
Planning	brain/planner, actions/contracts, permission-modes	Understanding to step contracts with expectations and risk classes
Execution	actions/engine, execution-log, recovery	Step execution, verification, bounded recovery, honest failure records
Agent path	engine/orchestrator, agent-loop, tool-registry, tool-catalog	Multi-step reasoning with 45+ tools; deny-list filtering
Device control	engine/desktop-controller, system-control, browser-automation, screen-vision	Apps, windows, input, browser pages, screenshots, vision
Knowledge	brains/device-brain, memory-brain, knowledge-graph, timeline, world-model; brain/device-index	Environment index, memory with importance, entity graph, event timeline
Providers	system/model-router, circuit-breaker, model-discovery, model-health	Failover chain, timeouts, model fallback, key management
Presence	renderer App, Companion, ChatLayout, SettingsPanel, theme system	Companion sprite, chatbox, settings, themes, approvals

Table 4-1 · Architecture layers and module ownership

Target-state changes

The target architecture keeps this spine and tightens five seams identified by the audit. Startup ordering moves window creation ahead of the blocking bootstrap so presence is immediate and the scan animation is real. The legacy planning trio, task-brain plus tool-executor plus capability-registry, is deleted in favor of the single live pipeline, removing approximately 1,200 lines of loaded-but-dead duplication. The three overlapping risk classification tables collapse into one source in the action contracts. The two unbounded JSON stores, the knowledge graph and the timeline, gain caps, decay, and debounced writes consistent with every other bounded store. The orchestrator's deterministic path gains the same per-step deadline enforcement the action engine already has, so no execution path can hang indefinitely.

Beyond repair-level tightening, the target architecture introduces three structural additions described in later chapters: a procedure store that turns verified multi-step plans into replayable, parameterized workflows (Chapter 15); a context assembler that produces compact, purpose-built model contexts from indexes and summaries instead of raw concatenation (Chapters 14 and 16); and a unified observation service that lets verification, screen understanding, and proactive checks share one screenshot-and-state

pipeline with shared caching (Chapter 13). None of these additions grows the process count or the storage footprint; each replaces repeated work with reusable structure, which is the chapter-2 thesis applied to the system's own architecture.

Boundary contracts

Architecture is its boundaries. The renderer-to-main boundary is the typed IPC registry: roughly sixty declared channels, one preload bridge, no renderer shell access, and a static contract test that fails CI when declaration, handler, and exposure disagree. The main-to-provider boundary is the model router's request contract: budgets declared, trail preserved, deadlines enforced. The engine-to-OS boundary is the tool registry: deny-list, risk classes, and verification expectations. Each boundary has a reason to exist, an enforcement mechanism, and a test, and the target state changes none of the boundaries themselves, only the quality of what crosses them.

Module ownership and the single-pipeline rule

Every capability belongs to exactly one subsystem, and the audit's dead-trio finding is the cautionary tale of what happens when ownership drifts: three modules kept alive by one unused import, parsed on every boot, executing never. The target-state ownership map is enforced by a simple CI check, an import graph walk that fails when a module under electron/ is unreachable from the entry point and not explicitly listed as test-only. The same review governs new integrations: a Skales-style capability lands as registry entries owned by the engine, never as a sibling executor. One pipeline, one registry, one risk table: duplication is not resilience, it is contradiction waiting to ship.

```
// Target import-graph rule (CI)
// every electron/**/*ts must be:
//   reachable from main.ts OR listed in test-only.allowlist
// violations fail the build
reachable(main.ts, glob('electron/**/*ts'))
  .difference(glob('electron/**/*ts'))
  .difference(testOnlyAllowlist) // must be empty
```

What does not change

Repair must not become redesign. The window factory, the execution log's ring pattern, the provider chain's honest trails, the close-to-companion semantics, and the test suite's contract-first shape are all correct and stay. The audit's fix list is deliberately conservative: reorder startup, delete the dead, bound the unbounded, guard the unguarded, surface the hidden. Architecture work that cannot name the evidence it answers and the test that will prove it is scope creep wearing an architecture costume, and this plan has no room for it.

The IPC boundary as a specification artifact

The IPC registry deserves its status as a first-class artifact because it is where architecture disagreements surface first. Every channel is declared once with its payload type, registered once in the main process, exposed once in the preload bridge, and consumed somewhere or deleted. The static checker that enforces this quadrilateral is extended, in the backlog, with renderer-usage coverage so a channel whose consumer disappears fails CI the same day. New integrations, Skales-style capabilities included, join this registry rather than inventing transport, which keeps the boundary auditable with a script instead of a meeting.

Data flow of one request, end to end

```
renderer
  chatSend(text) -> IPC CHAT SEND
main
  buildUnderstanding(text)      # brain/understanding
  resolveReferences(...)       # brain/references
  shape == direct ?
    actions/engine.execute(plan) # verified steps + log
  : shape == agent ?
    orchestrator.execute(goal)   # agent-loop + tool-registry
  stream/complete -> IPC chunks -> renderer render
  verification evidence -> execution log -> digest/learning
```

The flow above is the whole system in fifteen lines, and its value is that every box is a module with tests and an owner. The lifecycle chapter unpacks it; the audit chapter verified it; the roadmap changes parts of it without changing its shape. New engineers should be able to reproduce this diagram from the code in an afternoon, and the import-graph check exists precisely so the diagram and the repository cannot quietly diverge.

5. The Capability Model

5. The Capability Model

Quip's abilities are expressed as capability primitives: small, verifiable operations with typed inputs, typed observations, and known risk classes. Primitives compose into steps, steps compose into plans, and plans compose into procedures. The capability registry maps abstract goals to concrete implementations available on the current machine, so “open my email” resolves through a chain of questions, which mail client is installed, which browser is default, which account profile is signed in, rather than through a hardcoded answer. This indirection is what lets one understanding engine operate correctly on different machines without per-machine configuration.

Primitive families

Family	Examples of primitives	Verification signal
Application	launch, focus, close, list installed	process exists; window title/class observed
Window	move, resize, minimize, maximize, close	geometry within bounds; state flag observed
Input	click, type, hotkey, scroll, drag	post-action screen delta at expected target
Browser	open URL, search, read page, control tab	URL/title matched; content extract non-empty
Filesystem	open, create, move, rename, find, read	path exists; size/hash changed as expected
Clipboard	read, write	round-trip equality
Screen	capture, region OCR/vision, element query	capture succeeded; query returned within budget
System	volume, brightness, power state, network info	value read-back matches requested
Knowledge	device lookup, memory recall, web reading	answer grounded in cited source or index hit

Table 5-1 · Capability primitive families and their verification signals

Tool selection is a two-stage decision. Stage one is deterministic discovery: the understanding object's primary intent and target entities are matched against the device index and registry to enumerate concrete candidates, ranked by evidence such as default-handler status, recency of use, and the user's own phrasing history. Stage two, entered only when candidates are ambiguous or absent, asks the model to choose among the enumerated candidates, never to invent one; the model's output must name a candidate from the list or declare failure. This keeps model involvement bounded, auditable, and incapable of hallucinating applications that do not exist.

Local application versus web destination

A load-bearing distinction in the registry is between installed applications and web destinations, because users say the same word for both worlds. “Open VS Code” must resolve to the installed editor when present, never to a web search; “open YouTube” must resolve to the browser, never to a futile local search. The registry records for every known destination whether it is app-backed, web-backed, or both, and the resolver consults the device index first: an installed app with a default-handler claim wins for app-backed goals, a browser with account-aware URL construction wins for web-backed goals. When both exist, recency and the user's established pattern decide, and the choice is stated in the action's trust line so the user can correct the pattern, which is itself learned.

Registry entry schema

A capability registry entry is data, not code in a switch statement. Each entry binds an abstract capability to a platform implementation with the metadata the planner, gate, and verifier all need. The schema

below is the contract every future capability fills in; its completeness is what makes the rest of the pipeline mechanical, because planning, permission, and verification all read the same entry rather than private knowledge scattered across modules.

```
interface CapabilityEntry {
  id: string           // 'app.launch' | 'file.open' | 'web.read' ...
  risk: 'safe'|'medium'|'dangerous'
  input: JSONSchema     // typed, validated before execution
  observe(input): Observation // cheap deterministic evidence
  verify(input, obs, expect): Verdict
  timeoutMs: number     // per-step deadline
  destinations?: DestinationRef[] // app-backed / web-backed / both
}
```

Discovery ranking and evidence

When several candidates can serve a goal, discovery ranks them with explicit evidence rather than hidden preference weights. Default-handler status is the strongest claim; explicit install presence is next; recency of successful use follows; and the user's own vocabulary, the names they actually say for things, feeds an alias table that the understanding engine maintains. The ranking output is always visible in the plan's trust line, so “opened Edge because it is your default browser and VS Code is installed for code goals” is a sentence the system can actually produce. When evidence ties, the tie is surfaced as a quick clarification rather than resolved silently, which turns a potential wrong choice into a one-tap answer that permanently improves the alias table.

Fast paths as first-class citizens

A fast path is not an optimization bolted onto the pipeline; it is a registry-mediated resolution that completes without model involvement and still passes verification. The three fast-path sources are the device index for environment facts, the procedure store for repeated goal shapes, and session referents for conversational continuity. The execution log records which path served each task, and the fast-path ratio, deterministic completions over total tasks, is a first-class product metric with a target that only rises. A drop in the ratio is investigated like a regression, because it means the system forgot something it used to know.

The capability catalog in brief

Catalog area	Count (approx.)	Representative entries
Applications and windows	8	launch, focus, close, list, move, resize, minimize, state
Input synthesis	6	click, type, hotkey, scroll, drag, focus-field

Catalog area	Count (approx.)	Representative entries
Browser and web	7	open, search, read, tab-control, verify-landing, account-open, download
Filesystem and documents	8	open, find, read, write, move, rename, convert, organize
Screen and vision	4	capture, query-structured, interpret-vision, wait-change
System and environment	6	volume, brightness, power, network, foreground, clipboard-pair
Knowledge and reach	6+	device-lookup, memory-recall, web-read, repo/social channels

Table 5-2 · The capability catalog at a glance (registry is the source of truth)

The catalog's size is a floor, not a ceiling, and its growth rule is the composition rule: a new entry must be a primitive other steps can reuse, verified by observation, or it is a feature hack that the review rejects. The counts above approximate the audited registry; the exact, living number belongs to the registry file, and the capability map's status column tells the reader which entries are live today.

Destination families and their contracts

Destinations, the places goals land, are grouped into families with shared contracts: installed applications, web sites, account-scoped web properties, desktop documents, and system surfaces. Each family defines how launch targets are resolved, what verification observes, and what identity rules apply; account-scoped families add the profile-selection URL rules that keep “my mail” mine. A new destination joins a family and inherits the family's tests, which is why adding Spotify or a new search destination is config plus one hint-table entry rather than a new code path.

The fast-path decision matrix

Situation	Knowledge available	Path taken	Model role
Exact app named, installed	device index hit	deterministic launch + verify	none
Vague app name (“the editor”)	alias table hit	deterministic via alias	none
Vague, no alias	candidate list from index	rank, top if far ahead	none; else choose-from-list
Web destination	destination family rules	deterministic URL build	none
Compound objective	procedure match	replay adapted, verified	none on trusted replays
Genuinely novel multi-step	registry primitives only	agent decomposition	decompose, decide, interpret

Situation	Knowledge available	Path taken	Model role
Ambiguous reference	referents exhausted	clarify shape	draft the question if needed

Table 5-3 · When the model is called and when it is not

The matrix's purpose is to make model involvement boring: it happens exactly where the table says, for exactly the reason in the last column, and every other row is the fast path earning its name. The execution log tags each task with its row, and the ratio of deterministic rows is the system's efficiency scorecard. When a new feature request arrives claiming it needs a model call for row one or two, the request is wrong by definition, and the review says so with this table.

6. Environment and Device Understanding

6. Environment and Device Understanding

The Device Knowledge Layer is the system's map of the machine, and its defining discipline is diff-only maintenance. The first scan builds the index; every subsequent scan computes the current state, hashes it, and merges only differences, which is why steady-state lookups land under the 50-millisecond budget and rescans never block interaction. The index records the operating system, hardware summary, installed applications with launch identifiers, default browser and default handlers, displays and their geometry, and user-visible folders. Applications are discovered through the platform's own sources, registry and Start Menu entries on Windows, so the index reflects what the user actually sees in their own launcher.

Around the static index sits a small set of live environment signals: power state, network reachability, foreground application, and system load. The audit found the battery signal currently hardcoded to a constant, which silently disables low-power awareness; the fix, already scheduled in the roadmap, reads real values through the platform API with a short cache and degrades to an honest “unsupported” rather than a plausible lie. The workspace context tracker records the currently active project signals, such as recent files and open folders, in memory only, because it changes too fast to be durable and too sensitive to be stored casually.

Signal	Source	Freshness	Storage
Installed apps	OS registry / launcher walk	24 h TTL + diff on boot	device index JSON, bounded
Default browser, handlers	OS association query	diff on boot	device index JSON
Displays and geometry	Electron display API	event-driven	in-memory + window geometry file

Signal	Source	Freshness	Storage
Power state	platform API (real, not constant)	60 s cache	in-memory only
Network reachability	provider probe + system state	on demand	connection journal ring (80)
Foreground app	OS query	on task need	in-memory only
Recent files / projects	workspace context tracker	session-scoped	in-memory only

Table 6-1 · Environment signals: source, freshness, persistence

The environment layer exists to serve the capability model cheaply. Every deterministic resolution begins here; every planning step that touches the machine validates its assumptions here before acting. Its budgets are therefore structural, not aspirational: lookup latency is bounded by the index being in memory, rescans are bounded by diffing, and the index size is bounded by being derived knowledge rather than captured data. When Quip someday answers a question about the machine, it will almost always be answering from this map, not from a fresh interrogation of the operating system, and that distinction is where its speed comes from.

Diff-only maintenance, concretely

The diff discipline has four steps. Capture the current state from the platform source, compute a content hash per record, compare against the index's stored hashes, and merge only changed records with their timestamps. Applications gained or removed are the loudest diff; default-handler changes are the sneakiest and the most consequential, because they change where goals land. The scan runs at boot after the window exists and again on a day-scale time-to-live, and a manual rescan is always available in Settings but single-flighted, so an impatient user cannot stack five concurrent full scans.

Diff event	System response	User-visible effect
App installed	index gains launch id + handler claims	new app immediately openable by name
App removed	index marks absent; registry keeps knowledge	goal falls back to discovery or asks
Default browser changed	handler claim updated	web goals land in the new default
Display changed	geometry clamp recalculated	companion never lost off-screen
Nothing changed	no writes; boot proceeds	boot stays fast

Table 6-2 · Diff events and their consequences

Honesty when the environment cannot answer

Environment signals degrade gracefully and honestly. A battery read that the platform cannot provide returns unsupported, not zero; a network probe that both transports fail marks the transport as suspect in

the journal and says so in the Doctor; a foreground-app query that the OS denies returns unknown rather than a guess. Every planner step that consumes an environment signal declares what it does on unknown, proceed with reduced confidence or ask, so degraded data never silently upgrades into confident action. This is the same never-fake-success rule applied to telemetry, and it is why the battery fix in the backlog is specified as read-real-else-honest rather than read-real-else-assume.

Index record schema

```
interface DeviceIndexRecord {
  kind: 'app' | 'handler' | 'disolv' | 'folder'
  id: string          // stable launch id or association key
  name: string        // user-visible name
  aliases: string[]   // learned from the user's phrasing
  defaultFor?: string[] // goals this handles by default
  hash: string        // content hash for diff-only merges
  seenAt: number
}
```

The record's smallness is its feature: an index of thousands of entries stays tens of kilobytes, loads instantly, and answers the planner's questions without touching the operating system. Aliases are the learning surface, when the user says “the editor” and means VS Code, the alias lands with evidence and timestamp, and the next resolution is deterministic. The hash column is what makes rescans diff-only; records whose hash did not change are not rewritten, so even a full rescan is usually a handful of writes.

Rescan triggers and cadence

The index refreshes on four triggers: boot diff (always), day-scale time-to-live (opportunistic, background), explicit user rescan (single-flighted, visible in the Device tab), and invalidation events such as display changes for geometry. The cadence is deliberately lazy because the diff makes refreshes cheap and staleness expensive only for the records that changed. The one cadence rule with teeth is boot: the index must load from cache before the first task can resolve, and the boot budget in Chapter 21 holds it to that.

7. Screen Understanding and Computer Control

7. Screen Understanding and Computer Control

Computer control is the capability domain that makes Quip feel real, and it is engineered as a loop, not a button: SCREEN STATE is captured, a TARGET is identified, the ACTION is applied, and the result is VERIFIED by observing the new state. The observation service captures the screen through the platform's capture API, and two understanding paths exist over it: structured queries for specific facts, such as the foreground window's title and geometry, which are cheap and deterministic, and vision-model interpretation for open questions, such as what a dialog is asking, which are escalation-only because they cost tokens and time. Cheap checks always run first; vision is summoned only when the cheap checks cannot answer the plan's question.

Input synthesis, mouse and keyboard, is gated by the permission system's risk classes and always accompanied by post-action verification. A click step carries the expected consequence in its contract, for example a window title change or a menu opening, and the verifier checks that consequence before the step is marked complete. Targeting is resolved by description at execution time, the button labeled Submit, the second search result, the field containing the caret, rather than by brittle coordinates recorded earlier, which is what makes learned procedures portable across small layout changes. Coordinates are computed at execution time from the observed screen and never baked into procedures.

Risk class	Examples	Gate	Verification
Safe	read screen, query window state, read clipboard	auto-run in all modes	observation returned
Medium	launch app, open URL, type into focused field	auto-run in Standard and Dangerous modes; approval in Safe	expected state observed
Dangerous	delete files, send messages, purchases, system settings	approval in Safe and Standard; explicit allowlist in Dangerous	expectation contract + post-observation

Table 7-1 · Risk classes, gates, and verification (single source: action contracts)

Failure in the loop is a first-class outcome with a defined ladder: retry the action once if the state suggests a transient miss, re-resolve the target if the UI changed, degrade to a model-assisted decision if the screen is ambiguous, and finally cancel with an honest report of what was attempted, what was observed, and why the goal was not reached. The screen-understanding path never fakes a reading: if capture fails or vision returns low-confidence output, the report says so, because a companion that hallucinates its own screen is worse than one that admits it cannot see. The same observation service caches its captures within a task, so three verification checks in one plan do not cost three captures.

The observation service

Screen understanding, verification, and proactive checks share one observation service with one cache. A capture request names its region and its consumer; the service deduplicates within a task, honors a screen-change hint to invalidate, and prunes to a ring of eight files. Structured queries, window titles, geometry, process presence, run first because they are nearly free. Vision interpretation is an escalation with a declared confidence floor; below the floor the observation is reported as unclear, and the plan treats unclear as unverified. The service's cost ledger records captures and vision calls per task, which is how the efficiency thesis stays visible in the most expensive domain.

Input synthesis and target resolution

Mouse and keyboard actions resolve targets by description at execution time from the observed screen: the button with this label, the field with focus, the item at this position in this list. Recorded coordinates are forbidden in procedures because they break on the first DPI change or window move. Where description resolution is ambiguous, the verifier's post-check catches the miss and recovery re-resolves; where it is impossible, the step fails honestly instead of clicking the wrong thing confidently. The permission gate treats input synthesis as Medium by default and Dangerous when the described target is inside a web session where the consequence can leave the machine, an email send, a purchase, a post, because those are the clicks that matter.

Observation	Cost	Use
Window title / geometry	near-zero	launch, focus, close verification
Process presence	near-zero	app lifecycle verification
File / path state	near-zero	filesystem verification
URL / page title	low	browser navigation verification
Value read-back	low	settings verification
Screen capture	medium	target resolution, evidence
Vision interpretation	high	open questions only, confidence-floored

Table 7-2 · The observation cost ladder: cheap first, vision last

The vision call's contract

Vision is the most expensive observation, so it runs under a written contract. The request declares the question, the region if known, and the confidence floor; the model receives the capture plus a compact instruction to answer the question or return insufficient-confidence; the response is either a grounded answer that names what it saw or an explicit refusal, never a confident guess. The observation service logs every call with its question, floor, and outcome, and the cost ledger reports vision calls per task so an optimization, cropping to a region, reusing a capture, lowering the floor for a cheap retry, shows up as

savings rather than as folklore.

Why coordinates are computed, never stored

Stored coordinates are the classic computer-use trap: they work in the demo and die on the first display change. The architecture's rule is that procedures and plans carry descriptions, “the primary button in the confirmation dialog,” and resolution happens at execution time against the live observation. Resolution failure is honest, recovery re-resolves after re-observation, and repeated failure fails the step rather than clicking blind. The rule costs a few milliseconds per action and buys portability across displays, DPI changes, and localized UIs, which is the entire difference between a script and a companion.

Safety at the input boundary

Input synthesis is where software touches the user's world hardest, so the boundary has three absolutes. Inputs are always attributed, every synthetic action is logged with its step and plan. Inputs are always targeted through described resolution, never blind coordinates. And inputs are always permissioned, with the Dangerous class reserved for input whose consequence can leave the machine or destroy data. The audit verified the gate exists on the live path; the backlog adds the adversarial tests that try to bypass it, because a safety property without an adversarial test is an aspiration.

The screen pipeline, step by step

```
capture(region?) -> PNG (ring-buffered. hashed)
|
+-- structured queries first:
|   windowTitle / geometry / process / URL / value read-back
|
+-- if question remains open:
|   vision(question. region. confidenceFloor)
|       -> rounded answer | insufficient-confidence
|
+-- evidence -> verifier verdict (pass | fail | unclear)
```

The pipeline's order is its ethics: structured truth before model interpretation, and an unclear verdict rather than a guess at the bottom. Capture specifics are platform-honest, primary display by default, named region where the plan knows one, capture failure reported with the OS reason rather than retried into silence. The screenshot ring's pruning runs on write, so the storage contract holds even when a task crashes mid-verification.

Waiting for change

Many verifications are about change, the page finishing, the dialog appearing, so the observation service exposes wait-for-change: watch a region's hash or a structured value until it differs from the captured baseline or a deadline passes. The primitive is cheap, hash comparisons, and it replaces the naive sleep-and-hope pattern that plagues automation. Its timeout behavior feeds the recovery ladder the same

way any failed observation does, and its budget is part of the step's deadline, not an extra one.

8. Browser Control, Web Reading and Agent Reach

8. Browser Control, Web Reading and Agent Reach

Browser control is split by trust and capability. Quip's own browser automation operates pages it opens itself: navigating to URLs, filling known-destination search flows, reading page content for answers, and verifying that a navigation landed where the plan expected. It resolves web destinations through the account-aware URL builder, so signed-in Google properties open in the user's own account profile using the platform's profile-selection URL patterns, and it verifies by URL and title match after navigation. For pages the user is already viewing in their own browser session, Quip's default posture is observation, reading what is visible through the screen-understanding path, rather than control, unless the user explicitly grants an automation step.

Web reading is the pipeline that turns a URL into grounded, compact knowledge: fetch, extract the readable core, compress to the question-relevant span, and cite. It is the same pipeline used by the agent loop's reading tools and by verification checks that need to confirm what a page shows. The Agent Reach integration supplies the internet-access channel family, repository and social feeds among them, as registered tools with the same contract as local primitives: typed inputs, observed outputs, honest failure. The audit verified these channels are wired into the live tool registry, not merely present as files, and the roadmap keeps them there under the same verification discipline as everything else.

Integration boundaries

- **Licensing first.** External projects are inspected for license and architecture before any reuse; where direct code reuse is not legally appropriate, Quip implements a technically equivalent approach and says so in the capability map.
- **One registry, many channels.** Internet channels register in the same tool registry as local primitives; there is no second tool system for web work.
- **Health checks and fallback.** Every external channel has a probe; unhealthy channels are marked and routed around rather than retried into the user's waiting time.
- **Compound workflows.** The value of the integration is composability: a research task reads the web, writes a file locally, and opens the result, all as one verified plan composed from the same primitives.

The target posture for browser work, as for everything else, is that Quip may act broadly but claims narrowly. When it reports that it found something on the web, the report names the source; when it reports that it performed an action on a page, the report includes the observed post-state. Where a site's terms or technical barriers make a capability unreliable, the registry marks the destination as limited and the understanding engine steers around it, telling the user what it can and cannot do rather than attempting theater. That honesty requirement is not a legalistic nicety; it is what makes the browser domain trustworthy enough to automate at all.

Account-aware destinations

Web destinations that carry identity, mail, calendar, documents, open through the account-aware URL builder: the resolver reads the user's signed-in account order and constructs the profile-selecting URL form, so “my mail” lands in the user's own inbox rather than a chooser or, worse, a housemate's account. The builder is table-driven per destination family, with the encoding rules recorded as tests, because these URLs are exactly the kind of contract that breaks silently. Verification after navigation checks both URL shape and page title, and a miss reports which one disagreed, which is the difference between debugging a destination and guessing at one.

Web reading as a grounding tool

Web reading exists to ground answers, and its output format forces grounding: an extract carries the source URL, the fetched-at timestamp, and the relevant span, and any answer built on it cites that triple. When a fetch fails or a page is unreadable, the failure is reported with the HTTP or extraction reason rather than a shrug; when a page is dynamic to the point of instability, the registry marks the destination limited and the planner prefers a different evidence path. The same pipeline serves verification, confirming a page shows what the plan expected, and the agent loop's research steps, which is why it is a subsystem and not a script inside a feature.

Agent Reach channels under one contract

Every Agent Reach channel registers as a tool with the same shape as local primitives: typed input, observed output, deadline, and honest failure. Health probes run per channel and the Doctor reports each one's state; unhealthy channels are skipped by the planner rather than retried into the user's waiting time. The licensing rule stays simple: Quip reads public data through official or documented interfaces, never circumvents access controls, and records in the capability map which channels are live, partial, or blocked and why. Where a channel's terms change, the registry entry changes with it, and the change is visible in the release notes the same day it changes behavior.

Destination table excerpt

Destination	Kind	Identity	Verification
YouTube	web	none	URL + player title

Destination	Kind	Identity	Verification
Gmail / Google properties	web, account-scoped	profile-selecting URL	URL shape + account marker
ChatGPT / Gemini	web	session-scoped	URL + app chrome title
Spotify	app if installed, else web	none	process or URL
VS Code	app	none	window title + process
GitHub	web	user-scoped pages	URL + repo title

Table 8-1 · Destination families with identity and verification

Reading robustness

Web pages differ in politeness, and the reading pipeline grades its responses accordingly. Clean article structure extracts with the readable core; app-like pages fall back to a content heuristic and say so; pages that require interaction are marked limited rather than force-scraped; and failures carry their reason, status code, extraction empty, or render timeout, so the planner can choose another evidence path. The same grades apply to the verification use: confirming a search landed is cheap, confirming arbitrary page state is where vision escalates with its floor, and the boundary between them is the cost discipline.

9. Filesystem, Applications and OS Control

9. Filesystem, Applications and OS Control

The filesystem domain provides the same primitive discipline as every other control surface: operations are typed, risk-classed, and verified. Opening a file resolves through the OS association, moving and renaming return the resulting paths for the plan's next step to consume, and searching uses the file discovery index with the same diff discipline as the device index. Destructive operations, deletion and overwrite, sit in the Dangerous risk class and additionally respect a protective rule: paths outside the user's declared working areas require explicit approval regardless of permission mode, because a helpful assistant that can silently erase outside its sandbox is a liability, not a companion.

Application control covers launch, focus, and close, with verification appropriate to each: launch verifies a process or window appeared, focus verifies the target holds foreground, and close verifies the process exited or the window disappeared. OS-level controls, volume, brightness, and similar user-facing settings, register as Safe or Medium primitives with read-back verification, so setting volume to twenty percent confirms by reading the value back rather than trusting the call's return code. The clipboard is treated as a small, high-traffic primitive pair with round-trip verification; it is also the glue that lets otherwise

separate domains compose, for example copying a found paragraph into a document the plan just opened.

Operation	Risk	Verification	Guardrail
Open file / folder	Medium	association launch observed	none beyond gates
Create / write document	Medium	path exists with expected content shape	declared working areas
Move / rename	Medium	new path exists, old absent	collision check first
Search / read	Safe	results returned within budget	index-first, no full scans
Delete	Dangerous	path absent after action	approval + working-area rule
App launch / focus / close	Medium	process or window state observed	close confirms user-unsaved awareness where detectable
OS settings (volume, brightness)	Safe/Medium	read-back equals requested	range clamps
Clipboard read / write	Safe	round-trip equality	sensitive-data mask in logs

Table 9-1 · Filesystem and OS operations: risk, verification, guardrails

Document intelligence rides on these primitives: reading a document's structure, extracting a section, drafting into a new file, or exporting a converted copy are all compositions of read, write, and open primitives rather than a separate subsystem. This is deliberate. The moment document handling becomes its own engine, it grows its own failure modes and its own duplication; kept as compositions, it inherits verification, permission gating, and recovery from the underlying action engine for free, and every document task appears in the same execution log the user can audit.

Declared working areas

Filesystem power is bounded by declared working areas: the user's chosen project folders, downloads, and documents, recorded in settings and visible in the Desktop tab. Inside working areas, Medium operations run without ceremony in Standard mode. Outside them, every write-capable operation requires approval regardless of mode, and destructive operations require approval always. The rule is legible to a non-technical user, one sentence, no exceptions table, and it converts the filesystem from a source of anxiety into a workspace with edges. The audit's review of the document-tools path flagged exactly this class of risk, an arbitrary-path write guarded only by an extension check, and declared working areas are the structural fix.

File intelligence on top of primitives

Because reads, writes, moves, and opens are primitives with observations, file intelligence composes rather than forks. Summarizing a document is read plus model compression with the source cited; finding “the deck from last week” is index query plus temporal filter plus clarification when ties exist; converting a file is read, transform, write, with the write verified by existence and shape. The near-term

file-intelligence item in Chapter 29 adds semantic search, a capped local embedding store maintained by the same diff discipline, and nothing else, because the discipline is already here and the only honest new work is the index itself.

Composite task	Primitive composition	Verification chain
Summarize document	read + compress + render	source cited; span non-empty
Find file by description	index query + filter	candidates ranked with evidence
Convert format	read + transform + write	new path exists; shape validated
Organize downloads	query + move loop	moves verified; manifest reported
Backup folder	read-all + copy + hash	hashes match; report lists deltas

Table 9-2 · File intelligence as composition, not subsystem

Path safety rules

- **Working areas first.** Writes outside declared areas require approval in every mode; reads may roam but log.
- **No silent overwrite.** A write that would clobber an existing file either versions, asks, or fails loudly; the choice is the plan's, declared up front.
- **Trash over delete.** Where the platform offers a recycle path, destructive file operations use it, making most Dangerous file steps recoverable.
- **Symlinks are boundaries.** Operations do not follow symlinks outside the declared area, closing the classic escape hatch.
- **Manifest on batch.** Any multi-file operation produces a manifest in its result: what moved, what was skipped, why.

These rules exist because filesystem power is binary trust: one silent wrong write outweighs a hundred correct ones. With the rules in place, the Dangerous class in the filesystem shrinks to genuinely irreversible operations, and the approval panel's question, “Quip wants to delete three files in Downloads,” becomes rare enough that users read it when it appears. Rare questions with real teeth are the point of the permission design; frequent questions train users to click Allow.

10. The Task Lifecycle

10. The Task Lifecycle

Every request, from a three-word command to a compound objective, traverses the same lifecycle, and the lifecycle's stages are the system's real module boundaries. The stages are: request capture, intent understanding, context extraction, environment lookup, capability discovery, task decomposition, permission evaluation, plan creation, execution, observation, verification, recovery when needed, completion, and compact learning. The eleven-state task state machine in the codebase mirrors these stages so that the UI can always answer the user's real question, which is not “what stage is the code in” but “what is my companion doing and can I stop it.”

Stage	Owner	Output	Failure behavior
Request capture	renderer + IPC	raw message with session context	n/a
Intent understanding	understanding engine	structured understanding object	clarification gate asks, never guesses
Context extraction	understanding + context store	resolved references (“again”, “the file”)	missing referent surfaces a question
Environment lookup	device index	concrete candidates for targets	empty candidates trigger discovery or model assist
Capability discovery	capability registry	chosen primitive + implementation	no-capability yields honest refusal
Decomposition	planner	ordered steps with expectations	complex goals become verifiable sub-plans
Permission evaluation	permission modes + contracts	run / ask / refuse per step	asked steps park in WAITING_FOR_PERMISSION
Plan creation	planner	plan object logged to execution log	plan errors surface with cause
Execution	action engine / orchestrator	step results, observations	per-step deadline; recovery ladder
Verification	action verifier	expectation met or mismatch evidence	mismatch enters recovery, not success
Recovery	recovery module	retry / re-resolve / degrade / cancel	bounded attempts, honest trail
Completion	action engine	result summary + trust line	partial completion is stated as partial

Stage	Owner	Output	Failure behavior
Compact learning	procedure store	reusable procedure or memory update	learning never blocks the reply

Table 10-1 · Task lifecycle stages with owners and failure behavior

The lifecycle's economics differ radically by task shape, and that is the point. “Open VS Code” traverses all fourteen stages but does so deterministically in under a second: understanding resolves to an app-launch goal, the device index supplies the exact application, the plan is a single step with a window-observed expectation, and no model call occurs. The compound objective in Chapter 30 traverses the same stages but loops through decomposition, execution, and verification repeatedly, spending model attention only at genuine decision points. One architecture, two cost profiles, zero special cases: that is what objective-driven design buys, and it is why the system gets faster as its indexes and procedures fill rather than slower as its feature list grows.

The state machine the user actually sees

The eleven states are not internal trivia; they are the companion's vocabulary. Planning shows the character assembling; working shows effort; observing and verifying show the loop; waiting shows the approval ask; success and error flash honestly and briefly; cancelled shows respect for the stop button. The audit found one timing defect, a success pose that could stick past its window because no re-render was scheduled, and the fix is symbolic of the whole layer: even the emotions are on deadlines. The UI maps states from real engine events, never from timers guessing on the engine's behalf, which is what keeps the character's feelings truthful.

State	Trigger	UI expression	Timeout behavior
PLANNING	plan creation begins	assembling animation	none; bounded by planner
EXECUTING	step starts	working animation	per-step deadline drives recovery
OBSERVING	observation requested	attentive pose	capture deadline
VERIFYING	verifier running	squint pose	verifier deadline
WAITING_FOR_PERMISSION	gate asks	approval panel	no timeout; user owns this
RECOVERING	ladder engaged	focused pose	bounded attempts
COMPLETED / FAILED / CANCELLED	outcome	flash pose, 2 s, scheduled clear	pose always clears

Table 10-2 · Visible states with triggers and timeout behavior

Why one lifecycle is a product decision

A single lifecycle is what makes the companion legible. Users learn one grammar, request, watch, approve if asked, get an honest result, and that grammar holds whether the task is opening an editor or running a five-minute research plan. Every special case that bypasses the lifecycle, an unlogged action, an unverifiable claim, a silent shortcut, spends the same trust the grammar built. This is why the roadmap refuses feature work that dodges the spine: the spine is the product. Features are entries in its registries.

Micro-trace: “play it again”

```

1 request  'play it again'
2 understand intent=media.play. hidden=replay
3 references 'it' -> last media goal (Ariit Singh)
4 env lookup spotify installed -> app destination
5 capability app.launch + media control
6 permission medium -> auto (Standard mode)
7 plan 1 step: resume/search+play. expect player active
8 execute launch/focus: observe process+window
9 verify player title matches track -> pass
10 complete 'Playing Ariit Singh in Spotify again'
11 learn referent refreshed: no new procedure needed

```

Eleven lines, zero model calls, under a second: the micro-trace shows why the simple case is simple. The hidden-intent machinery and referent table carry the conversational weight; the device index carries the destination; the expectation carries the honesty. If any beat fails, Spotify missing, the track gone, the player silent, the same spine produces the honest branch rather than a shrug, and the user's next sentence has somewhere to land.

Micro-trace: the ask that saves a task

The second trace is a clarification done right. “Open the presentation” with two candidates, a .pptx from yesterday and one from last month, resolves to the clarify shape with the two names as options. The user taps yesterday's; the answer registers as a referent and nudges the alias table; execution proceeds and verifies. The cost was one tap and one frame of UI; the avoided cost was opening the wrong file, fumbling, and re-asking in free text. The clarification gate exists for exactly this exchange, and its design goal is that asking feels like attentiveness, never like incompetence.

Observability across the lifecycle

Stage	Event emitted	Surface that shows it
Understanding	understanding built (shape, confidence)	task chip; debug channel
Permission ask	WAITING_FOR_PERMISSION	approval panel; companion pose
Execution step	step started / ended with verdict	progress chip; execution log
Verification	observation + verdict	progress detail; log record

Stage	Event emitted	Surface that shows it
Recovery	ladder attempt	companion pose; log record
Completion	result + trust line	chat message; digest

Table 10-3 · Lifecycle events and the surfaces that render them

Every lifecycle stage emits an event, and every event has exactly one primary surface plus the log. This is why the user's question, what is it doing, always has an answer, and why the post-mortem's question, what did it do, has a record. The events are also the renderer's only source of task truth: the UI never infers state from timers or guesses, which is the discipline that keeps the companion's feelings synced to the engine's facts.

11. Understanding Engine

11. Understanding Engine

The understanding engine converts a raw message into a structured understanding object through a fixed pipeline of ten stages, each independently testable: normalization, language and register detection, intent classification, entity and target extraction, reference resolution, account and destination mapping, task-shape assessment, decomposition planning, clarification assessment, and confidence assembly. The understanding object carries primary, secondary, and hidden intents with confidence scores, resolved entities, the resolved conversational references, and the proposed task shape, direct, agent-assisted, or clarification-needed. Every downstream stage consumes this object; nothing downstream re-parses the raw text, which is what makes the pipeline provider-independent and test-replayable.

Reference resolution is where the system earns conversational fluency. “Play it again” must resolve “it” to the last media goal; “the second result” must anchor to a numbered list the system itself presented; “do the same thing in Chrome” must identify both the prior procedure and the browser override. The resolver maintains a small, session-scoped table of salient referents, recent goals, presented options, named files, and the heavy deliberate prompt instructs the model-assisted path to prefer these grounded referents over plausible inventions. When resolution fails, the pipeline returns clarification-needed with a specific question, because a wrong guess at “it” is more expensive than one short question.

The clarification gate formalizes when to ask versus when to act. The rule balances three signals: confidence of the understanding, cost of a wrong execution, and reversibility of the action. High confidence plus reversible actions proceeds; low confidence or dangerous actions ask. The gate's output is part of the understanding object, so the planner cannot accidentally execute a goal the gate flagged, and the UI renders the question as a normal conversational turn with quick-reply options when the ambiguity is enumerable. This keeps asking cheap and rare, which preserves the product feel: a companion that is

confident about the ninety percent and honest about the ten.

The understanding object, concretely

```
interface Understanding {
  primary: { intent: IntentId; confidence: number }
  secondary: IntentId[]
  hidden?: IntentId[] // e.g. 'verifv' implied by 'did it work?'
  entities: EntityRef[] // resolved targets with source evidence
  references: Resolution[] // 'it' -> last media goal. 'again' -> replay
  destination?: { kind: 'app' | 'web'; id: string; accountScored?: boolean }
  shape: 'direct' | 'agent' | 'clarifv'
  clarifv?: { question: string; options?: string[] }
  budget: { preferFastPath: boolean; allowModel: boolean }
}
```

The object is small on purpose. It fits in a log line, replays in tests, and travels across the IPC boundary without private fields. The heavy deliberate prompt that guides the model-assisted stages instructs the model to fill this object, not to answer the user; answers come later, from the planner and the execution trail. That separation, understanding as data, response as consequence, is what lets the pipeline be tested without network access and audited after the fact without guesswork.

Hint tables versus hardcoding

The intent parser carries hint tables, application aliases, site hints, verb mappings, and the audit notes their proper role: they seed discovery, they do not decide behavior. The line is mechanical. A hint may map a phrase to a candidate destination family; it may not decide the destination, because that is the registry's job on this machine. A hint may raise an intent's confidence; it may not create an action. Every hint-table entry is data with a test, and the parser's behavior for unseen phrasings remains well-defined: fall through to model-assisted understanding or clarification, never to a wrong confident action.

Failure of understanding is a result, not an error

When the pipeline cannot reach confidence, its output is the clarify shape with a specific question, and the lifecycle treats that as a successful understanding of an unresolvable request. The questions are engineered to be cheap to answer, enumerable options when the ambiguity is enumerable, a short free-text ask when it is not, and every answer feeds back: aliases learn, referents register, and the next occurrence of the same ambiguity is resolved without asking. Asking well is a skill the system is designed to improve at, which is the difference between a companion that feels attentive and one that feels interrogative.

Confidence calibration

Confidence is a number the system acts on, so it must mean something. The pipeline's confidences are calibrated against outcomes: an intent at 0.9 should be right about nine times in ten, and the calibration is

checked by the trail data, stated confidence versus actual verification result. Miscalibration is treated as a bug in the estimator, not as noise, because every threshold in the system, fast-path eligibility, clarification gating, approval defaults, sits on that number. The backlog's calibration task builds the report; the roadmap's later phases use it to tune thresholds with data instead of vibes.

Multilingual reality

The user base speaks Hinglish, English, and code, sometimes in one sentence, and the pipeline is built for it. Language detection informs register rather than routing: the same understanding object is produced whether the ask is “open VS Code karo” or “please launch the editor.” The voice chain's edge engine reads Hinglish naturally; the alias tables learn the user's own words for things; and the clarification questions render in the language the user is using. This is not a localization feature bolted on; it is the recognition that a companion's first courtesy is understanding you as you actually speak.

12. Planning, Permissions and Execution

12. Planning, Permissions and Execution

The planner turns a completed understanding into a plan: an ordered list of steps, each carrying its capability primitive, typed inputs, an expectation contract describing the observable postcondition, and a risk class from the single contracts table. Plans are built compositionally from the registry, so a plan step is never freeform text; if the registry cannot express a step, the planner either decomposes differently or declares the goal out of reach. Multi-step plans embed checkpoints, points where verification evidence is required before continuing, which bounds the blast radius of any wrong step in a long procedure.

Permission evaluation is mode-aware and step-aware. Three modes, Safe, Standard, and Dangerous-named for what they auto-approve rather than what they forbid, consult each step's risk class: Safe mode asks for every Medium and Dangerous step, Standard auto-runs Medium but asks for Dangerous, and the appropriately named full-trust mode still requires an explicit allowlist for genuinely irreversible operations. The approval surface is the in-chat panel with the step's plain-language description, its target, and its consequence, and a denial is a first-class plan outcome that the planner can use to reform, not a dead end. The audit found the mode machinery fully live in the engine and IPC but absent from the Settings interface, and closing that gap is a scheduled P1 product task, since a permission system without a visible switch is a permission system the user cannot actually govern.

Execution is the action engine's responsibility for direct plans and the orchestrator's for agent-shaped goals, but both obey the same discipline: per-step deadlines, structured results, observation evidence retained, bounded recovery, and a persistent execution log whose ring buffer holds the last 250 actions for post-mortems and for the learning stage. Recovery follows a fixed ladder, retry transient, re-resolve target, degrade to model-assisted decision, cancel honest, and cancellation is checked between steps and

inside long primitives, so a Stop press takes effect within one step, not after the plan. The distinction matters for trust: a companion that stops when told, even mid-plan, is one the user will actually let run.

Plan schema and checkpoints

```
interface Step {
  capability: CapabilityId
  input: ValidatedInput
  expect: Expectation // observable postcondition
  risk: 'safe' | 'medium' | 'dangerous'
  deadlineMs: number
  onFail: 'retry' | 'reResolve' | 'degrade' | 'cancel'
}

interface Plan {
  goal: string; steps: Step[]
  checkpoints: number[] // indices requiring verified evidence
  trust: { why: string } // plain-language rationale
}
```

Checkpoints are placed before irreversible or high-consequence steps, so evidence accumulates before the point of no return rather than after. The planner's composition rule is strict: steps reference registry capabilities only, and a goal the registry cannot express is either decomposed differently or declared out of reach, with the honest-refusal path tested as thoroughly as the happy path. Trust lines are mandatory, a plan without a why is not accepted by the engine, because the explanation is part of the product, not documentation of it.

The approval surface and remembered grants

The approval panel shows the step's description, target, and consequence in one card with two buttons and a lifetime choice: allow once, or allow this shape for this destination and remember it. Remembered grants live in the permission store, are listed and revocable in Settings, and are scoped narrowly, an email-send grant for one destination never becomes a generic send grant. The panel is rendered in-theme, in-position, and pausable, and a denial is delivered to the planner as a first-class outcome that may reform the plan, for example by choosing a safer destination, rather than treating the user's no as an error. The engine already implements the mode machine end to end; the scheduled work is the surface and the grants store, both specified here so the implementation has a contract to build against.

Permission mode semantics, precisely

Step risk	Safe mode	Standard mode	Full-trust mode
Safe (read, observe)	auto	auto	auto
Medium (launch, open, write in areas)	ask	auto	auto

Step risk	Safe mode	Standard mode	Full-trust mode
Medium outside declared areas	ask	ask	auto
Dangerous (delete, send, buy)	ask	ask	allowlist only
Irreversible + external effect	ask	ask	ask (never auto)

Table 12-1 · The exact mode-by-risk matrix the engine enforces

The matrix's last row is the trust boundary that survives every mode: some actions are never automatic, whatever the setting says, because the cost of asking once is trivial against the cost of one irreversible mistake. Mode semantics are enforced in one place, the action contracts' consumer, and the settings card renders this exact table in human language, so what the user reads is what the engine does, with no translation layer to drift.

Denial as input

A denial is not a dead end; it is information the planner consumes. The engine marks the step denied, the planner either reforms, choosing a safer destination, narrowing the scope, splitting the ask, or completes with the honest partial result. Repeated denials of the same shape teach the proactive layer to stop proposing it. This closes the loop that makes permissions conversational rather than obstructive: the system's response to “no” is adaptation, and its response to “always” is a remembered grant scoped exactly as wide as the user allowed.

Inside the action engine

The action engine's execution loop is small and strict: take the next step, check the cancellation token, validate the step's inputs against the capability's schema, enforce the gate, run within the deadline, observe, verify, record. Direct plans run here; agent plans reach here through the orchestrator's tool layer, which translates model decisions into registry calls so both paths share the same gate, deadline, and evidence contract. The shared contract is the design's quiet triumph: a step's safety does not depend on which planner proposed it, and the log cannot tell the difference, which is exactly the point.

Bounded recovery, budgeted

Ladder rung	Budget	Success signal
Transient retry	1 attempt	verification passes
Target re-resolve	1 fresh observation + resolve	target found; step proceeds
Model-assisted degrade	1 call, choose-from-registry	named action verifies
Cancel honest	0 (terminal)	report with full trail

Table 12-2 · Recovery ladder budgets

The budgets keep recovery from becoming its own failure mode, the retry storm that burns seconds and looks like effort. Each rung's cost is bounded, each rung's success signal is an observation, and the ladder's transcript lands in the step record for the digest and the post-mortem. When all rungs exhaust, the task fails with everything the user needs to decide what next, which is the ladder's real output: not rescue at any cost, but clarity at the end.

13. Observation, Verification, Recovery and Cancellation

13. Observation, Verification, Recovery and Cancellation

Verification is the system's honesty mechanism, and it is built on a strict principle: success is claimed only from observed state, never from the absence of an error. Each plan step's expectation contract names the observable it cares about, a window title, a URL, a file's existence, a value read-back, a screen region's content, and the verifier gathers that observable through the shared observation service. Cheap deterministic observations run first; vision interpretation is an escalation with an explicit confidence floor, and below that floor the step is treated as unverified rather than passed. The observation service also deduplicates within a task: multiple checks over an unchanged screen reuse one capture, which is both a performance measure and a token measure.

When verification fails, the recovery ladder engages in a fixed order with fixed budgets. A transient retry, once, handles missed clicks and slow loads. Target re-resolution handles UI drift, recomputing the described target from the fresh screen. Model-assisted degradation handles genuine ambiguity, asking the model to interpret the observation and name the next action from the registry's vocabulary. Cancellation with an honest report is the ladder's end, and the report names the step, the expectation, the observed state, and the attempted recoveries. The execution log records the whole trail, so a post-mortem is a read, not an investigation, and the user-facing summary is the same trail in plain language, because a companion that explains its failures is one the user can steer.

Failure shape	First response	Escalation	Report contains
Expected window/URL absent	short wait + recheck	re-resolve target from fresh observation	what was expected, what was seen
Click had no effect	retry once	re-resolve + alternative input path	target description + post-state
Ambiguous screen	structured query	vision with confidence floor	observation excerpt + confidence

Failure shape	First response	Escalation	Report contains
Primitive missing	registry discovery	model proposes from registry only	honest out-of-reach statement
Network/transport failure	provider failover	connection journal entry	provider trail with reasons
User cancels mid-plan	stop between steps	compensating action where safe	completed vs cancelled steps

Table 13-1 · Failure taxonomy and the recovery ladder

Cancellation is checked at every step boundary and inside long primitives via cooperative cancellation tokens, so Stop is honored within one step's deadline rather than after the plan completes. Partial completion is a legitimate outcome and is reported as such, listing what was done and what was not, because the alternative, silently claiming success or silently continuing, is exactly the behavior the trust contract forbids. These mechanics also serve the future: the same verifier and ladder will verify learned procedures on replay, which is what makes workflow learning safe rather than reckless.

Expectation contracts in practice

An expectation names its observable, its match rule, and its timeout. Match rules are deliberately simple: equality for values, existence for paths, containment for titles, delta for screens where change itself is the evidence. Complex predicates are a smell, because a verifier that reasons like a planner is a second planner; when a check needs complexity, the plan decomposes so each check stays simple. Contracts are data, so the execution log can replay them after the fact, and a post-mortem shows exactly what success was defined to mean, which retires the phrase “it said it worked” from the project's vocabulary.

Cancellation semantics

Cancellation is cooperative and bounded. Every step receives a token checked at entry, at observation, and inside long primitives' poll loops; the Stop control sets the token, and the engine stops at the next boundary within one step's deadline. Completed steps stand, the trail is reported as done-versus-not-done, and compensating actions, closing a page the plan opened, are offered when safe and cheap. A cancelled task is never a failed task in the logs; it is its own outcome with its own count, because users who stop things are using the system correctly and the metrics should say so.

Verification target	Match rule	Timeout	On mismatch
Window appears	title contains goal token	5 s	re-resolve then honest fail
URL landed	URL shape + title	8 s	report which disagreed
File exists	path existence	2 s	list near-miss candidates
Value set	read-back equality	3 s	report both values

Verification target	Match rule	Timeout	On mismatch
Screen region	structured query or floored vision	10 s	unclear counts as unverified
Process exited	absence	6 s	report still-running with pid

Table 13-2 · Match rules, timeouts, mismatch handling

Evidence schema in the execution log

```
interface StepRecord {
  step: number; capability: string
  startedAt: number; endedAt: number; deadlineMs: number
  expectation: string // human-readable postcondition
  observation: { kind: string; value: string; at: number }
  verdict: 'pass' | 'fail' | 'unclear' | 'cancelled'
  recovery?: { attempt: string; outcome: string }[]
}
```

Every field earns its bytes in a post-mortem: the verdict's four states, including unclear, keep the verifier honest; the recovery array is the ladder's transcript; and the timestamps make the deadline stack's behavior auditable after the fact. The ring of 250 records bounds the file, and the digest surface reads the same records the post-mortem would, which is how the weekly digest and the post-incident review share one truth source.

The unclear verdict

Unclear is the verifier's most important output and the one naive implementations omit. When an observation cannot decide, a capture came back ambiguous, a vision answer fell below its floor, a title partially matched, the verdict is unclear and the plan treats it as unverified: recovery gets one more observation with a different cheap check, and if ambiguity persists, the step fails honestly rather than passing hopefully. Unclear is what prevents the worst failure class in computer use, the confident wrong claim, and its presence in the schema is a design stance, not an accident.

14. Memory: Compact Knowledge, Not Bloated History

14. Memory: Compact Knowledge, Not Bloated History

Memory in Quip exists to change future behavior, and that purpose sets its shape. Facts, preferences, and stable entities are extracted from conversation by a periodic, model-assisted pass and stored with an importance score; scores decay, low-value entries are pruned automatically, and hard caps bound the store regardless of decay. The audit verified the discipline is mostly present, a 200-entry working cap with importance pruning, and found the exceptions that matter: the knowledge graph and the timeline grew without bound and rewrote their files synchronously on every event. Both stores are being brought under the same contract as the rest, caps, decay, and debounced writes, because one unbounded file is a slow storage leak wearing a useful name.

What never gets stored is as important as what does. Full conversation transcripts persist only in the renderer's session archive under the user's control; the main process keeps summaries and extracted facts, not raw dialogue. Screenshots live in a ring of the last eight and only while vision tasks need them. Workspace context is in-memory, session-scoped, and never written to disk. API keys live in the user-data environment file with masked logging, and the audit's security review confirmed keys never reach the renderer or the logs. The result is a memory system whose entire permanent footprint is measured in a few megabytes and whose growth is governed by importance, not by time.

Store	Cap / bound	Decay or eviction	Write pattern
Memory facts	200 working entries	importance score + decay + prune	debounced 2 s
Knowledge graph	bounded entities and links (fix scheduled)	recency + importance	debounced (was sync per event)
Timeline events	bounded window (fix scheduled)	age-based	debounced (was sync per event)
Execution log	ring 250	ring overwrite	debounced
Connection journal	ring 80	ring overwrite	debounced
Vision captures	last 8 files	prune on new capture	on demand
Chat sessions	20 archived sessions x 60 messages	user-clearable	on message change

Table 14-1 · Every persistent store with its bound and write pattern

The compact-memory principle also governs what reaches a model. Prompt assembly draws from memory by relevance, the relationship profile, the current task's entities, a bounded slice of recent conversation gist, never the raw archive. This is the capability-per-token thesis applied to memory: a fact that changes a decision is worth its bytes, a transcript that merely happened is not, and the architecture draws that line mechanically so it does not depend on anyone's daily judgment.

Importance, decay, and the pruning conversation

Importance is scored from evidence: repetition across sessions, explicit pinning, use in successful task resolution, and user correction, which teaches fastest. Decay is gentle and exponential; pruning runs automatically below a floor and never deletes pinned or recently used entries. The user's Memory tab is the conversation's other half: every entry is viewable with its score, pinnable, forgettable, and the prune button reports how many it removed and why. Memory the user cannot see is memory the user cannot correct, and uncorrectable memory drifts from useful to creepy in ways no metric catches.

The two stores being brought to heel

The knowledge graph and the timeline earn their keep only under the same contract as everything else. The graph's fix is a bound on entities and links with recency-and-importance eviction, plus debounced writes; the timeline's fix is an age-based window with the same write pattern. Both fixes are scheduled with soak tests, run a thousand upserts, assert the file neither exceeds its bound nor loses its newest entries, and both carry a migration that trims existing files honestly, reporting what was removed. Bounding a store after the fact is a small, delicate operation, and the delicacy is exactly why it is specified here rather than improvised.

What the model sees

Prompt assembly draws from memory with a relevance filter and a declared budget: the relationship profile's current summary, the entities this task touches, a bounded slice of recent conversational gist, and the device facts the goal implies. Raw transcripts never enter a prompt; the archive is the user's, not the model's. The assembler's cost ledger reports tokens by block, so the prompt audit of Chapter 2 is a query, not an archaeology, and a prompt that consistently pays for blocks that do not change answers is treated as a bug in the assembler, not a cost of intelligence.

Extraction policy

The extractor's mandate is narrow: facts and preferences that plausibly change future behavior, stable entities with names the user uses, and the relationship profile's evolving summary. It explicitly does not store plans of the day, transient moods, one-off references, or anything the user would be surprised to see in a list. Every extraction carries its evidence span, so a memory can show where it came from, and disputed memories are correctable by the user, with the correction outranking future extractions on the same topic for a grace period. The policy's test is embarrassment: if an entry would embarrass the system in front of the user, it does not belong.

Store growth regression tests

The bounding contract is enforced by tests that age stores artificially: a thousand extractions against the memory cap assert the oldest unimportant entries evicted; a thousand graph upserts assert the entity and link bounds; a thousand timeline events assert the window. The tests run in the sandbox on every build, which means a regression that reintroduces unbounded growth fails CI within minutes of being written, not after a user's month of usage. Bounding as a tested invariant is what makes the frugality NFR real.

15. Workflow Learning and Procedural Knowledge

15. Workflow Learning and Procedural Knowledge

Workflow learning converts verified multi-step executions into reusable procedures, and it is the mechanism by which Quip becomes faster and cheaper the longer it is used. A procedure records the goal shape, the parameterized step sequence, the expectation contracts, and the resolution hints, described targets rather than coordinates, so that replay adapts to minor UI drift and different file names. Learning is conservative by design: only plans whose every step verified cleanly, or whose recoveries are themselves recorded and understood, are eligible; a procedure that needed the model to guess twice is stored as notes, not as an executable recipe.

Replay is a first-run under supervision, not a blind macro. The stored procedure executes through the same action engine with the same verification and the same permission gates; if an expectation fails, the normal recovery ladder engages and, when it exhausts, the system falls back to planning the task fresh rather than forcing the procedure through. Successful replays increment the procedure's trust score, which affects how the scheduler uses it, trusted procedures become deterministic fast paths for their goal shape, meaning zero model tokens for repeat work, while unproven ones remain suggestion-only. The user can inspect, pin, or delete procedures in Settings, because a learned habit the user cannot see is a habit the user cannot trust.

The storage cost of a procedure is deliberately tiny, typically under two kilobytes of structured JSON, and the registry de-duplicates goal shapes so that fifty variations of “open the site and search for X” collapse into a handful of parameterized procedures. This is the capability-per-byte thesis at its most visible: the system's growing skill is measured in hundreds of small records, each of which eliminates a whole class of future model calls. Learning is the compounding asset; everything else in the architecture exists to make sure the asset stays verifiable, inspectable, and small.

Procedure record schema

```
interface Procedure {
  goalShape: string          // normalized goal pattern
  steps: ParameterizedStep[] // described targets, never coordinates
  expectations: Expectation[]
  trustScore: number         // replay successes minus failures
  lastVerified: Timestamp
  source: 'learned' | 'user-authored'
  sizeBytes: number          // budget: < 2 KB typical
}
```

The schema's discipline is that it contains no screen captures, no raw transcripts, and no coordinates: a procedure is a recipe, not a recording. Parameterization covers names, folders, destinations, and account scope, so one learned pattern serves many instances. Trust scores gate usage: trusted shapes become fast paths, unproven ones suggest only, and three consecutive replay failures retire the procedure to notes pending re-learning. All of this is visible in Settings, where procedures can be inspected, pinned, edited, or deleted, because a learned habit must remain the user's possession.

Learning from failure too

Failed plans teach differently: they record the divergence, the step where observation disagreed with expectation, and the recovery that worked or did not. That record feeds two consumers, the planner's risk estimates for the goal shape, and the understanding engine's referents, so “not that one, the other one” has somewhere to land. Failure records are bounded like everything else and age out; they exist to make the second attempt better, not to keep a museum of mistakes. The learning stage never blocks the reply, because reflection is a background craft and the user's answer is already on screen.

Replay adaptation rules

Drift observed at replay	Adaptation	If adaptation fails
Window title varies slightly	fuzzy match on goal token	re-resolve from fresh observation
Target moved on screen	description resolution at new location	recovery re-resolve
Destination account differs	account-aware URL rebuild	clarify once, remember
Step's object absent	search for described equivalent	honest fail; procedure flagged
Extra dialog appears	dismiss if safe and known	degrade to model decision

Table 15-1 · Replay adaptation: drift, response, escalation

Adaptation is bounded by the same permission gates as first execution: a replay never takes an action class the original plan did not earn. Trust scores update from replay outcomes, and a procedure that adapts successfully three times graduates to fast-path eligibility for its exact shape, which is the moment

the user stops paying anything for that chore at all.

User-authored procedures

The procedure store accepts authorship from both directions: learned from verified runs, and authored by the user, a recorded sequence of steps the user demonstrates or describes. Authored procedures carry the same schema, the same verification, and the same permission gates; authorship only changes the trust prior, user-authored starts trusted for its stated shape but still verifies every replay. The Settings surface lists both kinds identically, because from the user's chair a habit is a habit regardless of who taught it.

16. Model Routing and Provider Abstraction

16. Model Routing and Provider Abstraction

The model router is a failover chain, not a favorite: providers are ordered by user preference and health, every request walks the chain, and the trail of what was tried and why it failed is preserved end to end. The audited chain is Groq, Gemini, Cerebras, NVIDIA, OpenRouter, and Ollama as the keyless offline resort, with per-provider model fallback when a model identifier is rejected, a circuit breaker that parks failing providers with exponential backoff and respects rate-limit retry hints, and transport selection between the Electron network stack and direct fetches for proxy-hostile environments. Timeouts are explicit at every layer: connection, first byte, overall, and per-stage tool budgets, because a chain without deadlines is just a slower single point of failure.

Provider abstraction extends to capability. Vision-capable models are flagged in the registry so screen-understanding requests route only to providers that can actually see; tool-use capable models are likewise distinguished for the agent loop. Token budgeting is part of the router's contract: prompt assembly declares its budget, the assembler fills it with the most relevant compact slices, and oversized contexts are summarized rather than truncated mid-structure. The router also tracks per-provider latency and health into the Doctor screen, which turns what used to be mysterious chat failures into a readable report the user can act on.

Provider	Role in chain	Distinguishing capability	Key source
Groq	primary by default	fast completions; neural TTS; vision models	gsk_ key
Gemini	high free quota	vision + long context	AIza key
Cerebras	speed alternative	fast completions	csk- key
NVIDIA	capacity alternative	hosted open models incl. vision	nvapi- key

Provider	Role in chain	Distinguishing capability	Key source
OpenRouter	breadth fallback	model variety incl. free vision	sk-or- key
Ollama	offline last resort	local, keyless, opt-in	none (local app)

Table 16-1 · The provider chain and each link's role

The routing philosophy is cheap-and-fast before deep-and-expensive at the task level, not just the provider level. Deterministic paths bypass the router entirely; classification-scale decisions prefer small fast models; only genuine reasoning, decomposition, and screen interpretation earn large-context calls. This internal market is what keeps the efficiency thesis honest: the router's metrics report tokens per task over time, and a rise without a corresponding capability rise is treated as a regression to be engineered away, not as the cost of doing business.

Timeouts and the deadline stack

Layer	Deadline	On expiry
Connect	12 s	transport fallback, then next provider
First byte	8-10 s (streaming chat)	next provider; breaker notes failure
Stream stall	25 s between chunks	stream fails with honest trail
Overall completion	60 s chat; 45 s tools	task-level recovery or cancel
Plan step	per capability entry	recovery ladder
Boot probe	8 s total, parallel	degraded-start flag, Doctor note

Table 16-2 · The deadline stack at every layer

The deadline stack exists because every layer that lacks one becomes the new longest wait. The audit's finding that the deterministic orchestrator path lacked the step deadline the action engine had was exactly this pattern, one unguarded layer quietly owning the user's patience. With the stack complete, the worst case is bounded and legible: the trail names the layer that spent the time, which converts “it hung” into a fixable fact.

Token budgeting in the assembler

The assembler treats tokens as a budget with line items: system identity, task-relevant memory, device facts, conversation gist, and tool schemas for the agent path. Each line has a cap and a relevance rule, and overflow is resolved by compression of that line, summarizing the gist, narrowing entities, not by truncating the assembly mid-structure. The agent loop's tool schemas are the largest line and the most cacheable, so they are rendered once per session and reused. The budget is visible in diagnostics as tokens-per-block per task, which makes cost regressions a query instead of a debate.

Model fallback inside a provider

Provider failover is the outer ring; model fallback is the inner one. When a provider rejects a model identifier, deprecated, decommissioned, or unavailable on the account's tier, the router tries the provider's spare list before moving to the next provider, because the provider may still be the fastest healthy path. Spare lists are data with health checks, and the audit-era lesson is recorded in the environment template itself: a decommissioned free-tier model must never strand the whole provider. Fallback events are counted per provider and surface in the Doctor, because a provider that quietly runs on spares is a provider the user should know about.

Health states and what each does

Provider state	Entered by	Routing effect	Clears by
healthy	successful call	normal order	n/a
degraded	slow but successful	normal order; Doctor notes latency	fast success
parked	2 consecutive failures	skipped with backoff	backoff expiry + probe
rate-limited	429 with Retry-After	skipped until hint expiry	hint expiry
misconfigured	bad key format / rejected model	skipped; Settings flags it	user fixes key
offline-opt-in	Ollama unreachable	last-resort only	service returns

Table 16-3 · Provider health states, effects, and exits

The states make the chain's behavior legible: a user reading the Doctor sees not just which providers work but why each one is where it is, with the exit condition named. Parked providers that recover on probe rejoin silently; misconfigured ones stay flagged until the user acts, because pretending a bad key is a network hiccup helps nobody. The breaker's memory is per-provider and short-lived by design, so transient incidents self-heal without manual intervention.

Choosing providers deliberately

The router's default order is sensible but personal: a user who prefers a specific provider's voice or speed sets the primary, the chain respects it, and the fallback order remains automatic below it. Enabled-flags let a user exclude a provider entirely, keys configure the rest, and the Doctor explains the resulting chain in one sentence per provider. The design goal is that the user never needs to learn the word failover to benefit from it, and never has to debug it when the Doctor can narrate it.

The routing loop, pseudocode

```
for provider in chain():           # user-preferred + health-ordered
    if breaker.isParked(provider): continue
```

```
for model in [configured, *snares(provider)]:
    try:
        return call(provider, model, budget, deadline)
    except Retrvable as e:
        breaker.note(provider, e): continue
    except Fatal as e:
        trail.append(provider, reason): break
raise ModelTransportError(trail) # honest, complete, manned to IIT
```

The loop is the whole routing policy in ten lines: health ordering from the breaker, model spares inside each provider, a trail that survives total failure, and an error type the chat layer maps into the calm, actionable message the renderer already knows how to show. Every knob the env template exposes, primary provider, enabled flags, transports, budgets, plugs into a named point in this loop, which is why the configuration story and the code story never diverge.

17. Voice and Multimodal Interaction

17. Voice and Multimodal Interaction

Voice output is live and layered: a Groq neural voice leads, a free online neural voice with strong Hinglish handling backs it, and the platform's local voice closes the chain offline, with automatic failover between engines mid-conversation. The audit confirmed the chain works and identified the experience gap between functional and delightful: replies are spoken whole rather than sentence-streamed, so long answers begin with latency. The roadmap's voice stream work pipes sentence-level synthesis to the renderer as the model streams text, which collapses perceived latency to roughly the first sentence's synthesis time, the same transformation streaming did for text.

Voice is integrated with presence rather than bolted on: the companion's speaking state drives a visible animation, a top-bar indicator reflects actual audio playback rather than intent, and a test surface in Settings speaks a sample through the current engine so users can tune voice without leaving the app. Autoplay policy is handled honestly, the first user gesture unlocks audio, and the mute state is a real setting, not a hidden toggle. Voice input, the microphone side, is scoped as a near-term roadmap item with the same discipline: local speech-to-text options evaluated for privacy, provider-backed options for accuracy, and a push-to-talk default so an always-listening companion is an explicit user choice rather than an ambient assumption.

Multimodal interaction beyond voice follows the same integration pattern. Screen vision is already the template, an escalation path with confidence floors and honest failure, and future inputs, picking a file by drag-drop into the chat, referencing an image on the clipboard, asking about a selected region, are extensions of the observation service and the understanding engine rather than new subsystems. The design rule across all of them is that inputs compose into the same understanding object as text, so the

lifecycle in Chapter 10 remains the single spine no matter how the request arrived.

The voice chain's failure semantics

Voice failure is quiet by design but never silent in the record: if the neural engine fails, the edge engine takes the sentence; if both network voices fail, the local engine speaks; if all three fail, the reply renders text-only with a one-time note, not an error modal. The speaking indicator reflects actual audio output, driven by playback events rather than intent, and a stop control exists wherever a speak does. Settings exposes engine choice, voice choice, and a test button, and the whole chain is exercised by tests at the contract level, engine selection given each failure matrix row, so the fallback story is code, not folklore.

Voice input's privacy posture

Voice input is scoped before it is built precisely because it is an ambient-risk surface. The default is push-to-talk, never always-listening; audio is captured only while the control is held; and the transcription path follows the same provider abstraction as text, local-first options evaluated for privacy, provider options for accuracy, with the chosen path named in settings. Recordings, if any temporary buffer exists, are ring-bounded and never persisted. The feature ships only when its off state is as trustworthy as its on state, because a microphone is where companion software earns or loses a kitchen-table conversation.

Voice capability	Status	Infrastructure	Horizon
Neural TTS with fallback	live	3-engine chain	shipped
Sentence-streamed speech	planned	chunked synthesis + audio queue	near
Push-to-talk input	planned	capture + ASR abstraction	near
Per-companion voice identity	planned	voice profile per companion	medium
Full duplex conversation	long-term	echo-cancelling capture + barge-in	long

Table 17-1 · Voice roadmap with infrastructure pricing

Audio lifecycle and states

```

TDI F -> SPEAKING(queue) -> DONE
  on engine fail: SPEAKING -> FALLBACK(next engine) -> SPEAKING
  on stop: SPEAKING -> STOPPED (immediate)
  on new reply: queue replaced (latest wins)
renderer: indicator driven by real playback events
settings: engine select. voice select. test-speak button

```

The lifecycle's rule is latest-wins with an immediate stop, because a companion that keeps talking over the user is worse than one that says nothing. The indicator's honesty requirement, driven by playback events rather than intent, survives the fallback path: when the edge engine takes over mid-sentence, the

user sees continuous speaking, and the log shows the engine switch. The test-speak button exercises the whole chain, which turns voice configuration from guesswork into a five-second check.

Voice identity per companion

The medium-term voice identity work gives each companion a distinct voice profile, pitch, pace, and engine choice, drawn from the same config module that owns their colors and personalities. The profile is data, so tests can assert every companion has one and every profile renders on the chain; and the user can override any companion's voice, because personality is the user's to tune. The design keeps identity in configuration rather than in code paths, which means adding a seventh companion someday is an entry, not a feature.

18. The Six Companions

18. The Six Companions

Six companions ship as first-class characters, each with a distinct body, palette, aura, and personality line: Pix the Creative Spark, Kai the Wise Guide, Ren the Fearless Explorer, Bubbles the Joyful Friend, Cappy the Calm One, and Skales the Original Gecko. The audit verified that each has hand-drawn sprite art rather than a recolored template, and flagged the two gaps that matter for polish: two companions accepted a cursor-tracking parameter their art ignored, making eye contact inconsistent across the roster, and one progression surface hardcoded one companion's brand colors for all six. Both are scheduled polish fixes; neither is a redesign.

Companion behavior is state-driven: idle, hover, thinking, responding, working, planning, observing, verifying, waiting, success, error, and cancelled all have distinct animation treatments, with an energy signal from the mood engine scaling animation tempo and a blink loop plus randomized micro-gestures keeping the character alive between events. Cosmetics unlock through the progression system as relationship depth grows, tiered accessories and auras rendered as sprite add-ons, giving the companion system a gentle long-game without paywall mechanics. The mood engine's inputs, time of day, interaction recency, task outcomes, are deliberately few and honest; a companion that performs emotions it has no signal for reads as fake within a week.

Companion	Personality	Signature visual	Signature behavior
Pix	Creative Spark	antenna spark, cyan-pink aura	sparkle burst on task success
Kai	Wise Guide	star antenna, indigo palette	slow blink while thinking
Ren	Fearless Explorer	gold-on-black explorer palette	quick eager hop on new goals
Bubbles	Joyful Friend	wobble-blob body, puddle shadow	bounce loop when responding

Companion	Personality	Signature visual	Signature behavior
Capy	The Calm One	barrel body, warm earth palette	unhurried yawn on idle
Skales	Original Gecko	leaf sprout, animated tail curl	tail flick on notification

Table 18-1 · The companion roster: identity, art, behavior signatures

The companion is the product's emotional interface, and the discipline that keeps it credible is that its states are driven by real system state: it plans when the planner plans, verifies when the verifier verifies, and celebrates only when verification passed. That wiring, not the art, is why the character feels alive rather than decorative, and it is also why the companion doubles as an honest status indicator: a user who learns to read the sprite has learned to read the task engine.

Consistency as a roster contract

A six-character roster is consistent when a user can close their eyes and name the speaker, and open their eyes and see the state. The contract: every companion implements every state, every companion tracks the cursor, every companion's cosmetics exist at the same tiers, and every companion's colors come from the config module, not from art-side literals. The audit's two gaps, two companions ignoring the eye-offset parameter and art-side hex literals outside the config, are polish tickets against exactly this contract, and the acceptance test is a rendered pass across all six at all states, eyeballed once and automated thereafter for structure.

The mood engine's inputs

The mood engine keeps its inputs few and honest: time of day, recency and tone of interactions, task outcomes, and energy from system load. Its output is a small set of scalars, energy, warmth, playfulness, that scale animation and select idle gestures; it does not write prose about feelings, because performed emotion without signal is the uncanny valley. The progression system reads the same signals to unlock cosmetics, which keeps growth and mood telling one coherent story: the relationship deepens through real use, and the sprite shows it.

Why states must be true

The companion's states are driven by the task engine's events, and the audit's stuck-pose finding shows why that wiring is a contract and not a convenience: a character that celebrates for eleven seconds when the plan said two is a character that lies a little, on a schedule nobody chose. With the pose-clear timer fixed, every visible feeling has a start event, an end event, and a deadline, which makes the sprite as testable as the engine it mirrors. Presence that tells the truth is the entire differentiator between a companion and a widget.

Animation spec, per state

State	Motion signature	Tempo source	Clears on
idle	float + blink + rare micro-gesture	mood energy	n/a (baseline)
thinking	eyes up-left, mouth glow pulse	mood energy	first token / plan start
working	lean-in loop with tool tint	task progress	step outcome
verifying	squint + scan line	verify start	verdict
waiting	hands-behind pose, approval glance	n/a	grant or denial
success	hop + sparkle burst	fixed 2 s	scheduled clear
error	droop + single drip	fixed 2 s	scheduled clear

Table 18-2 · Animation signatures: every state with a tempo and a clear condition

The spec's discipline is the right-hand column: every non-baseline state has a clear condition, event or deadline, so no expression sticks. Tempo comes from the mood engine's energy, which is how the same animation reads calm at midnight and bright at noon without a second rig. The roster contract requires all six bodies to implement all seven signatures plus the transit poses, and the audit's consistency pass checked exactly that matrix.

Cosmetics as earned identity

Cosmetics exist to mark a relationship, not to gamify attention: three tiers, small accessory at depth, signature item later, aura at the top, each tied to progression milestones the user can see in the Progression tab. Cosmetics never change behavior, never gate features, and never nag, they are the quiet reward for a companion that has actually been useful. The implementation detail that matters is that cosmetics are config-driven overlays on the sprite rig, so a new cosmetic is an entry and a test, not a redraw.

19. Desktop Presence, Chatbox and Themes

19. Desktop Presence, Chatbox and Themes

Presence is architectural, not cosmetic. The single window factory renders four modes: companion-only, a small anchored panel, the full application, and true fullscreen, each with defined geometry, anchoring, and transition behavior. The close button's semantics are fixed by contract: it returns the window to companion mode and never quits; the mascot remains on screen until the user chooses the explicit Quit path in Settings, and a second launch of the app focuses the existing instance instead of spawning a rival. Window position persists, drag works per-pixel through the window-move channel, and the spatial brain clamps geometry so the character can never be dragged off-screen.

The chatbox is a real opaque surface, the audit confirmed the desktop cannot bleed through the panel, full, or fullscreen layouts, and its composition includes the loading, streaming, error-with-retry, task-progress, and approval states a real product needs. Branding lives where users look: the logo anchors the chatbox's top-left corner and adapts to theme colors, and the companion's task chips tint with the active character's palette. Ten themes, Quip Violet as brand default plus nine from light to carbon, apply through CSS custom properties with pre-paint application so the chrome is never wrong on boot; the audit's theme findings, a handful of hardcoded status colors and one component ignoring themes entirely, are enumerated as polish tasks with token-mapped fixes.

Mode	Geometry	Presence	Primary use
Companion	132 x 176 sprite, always-on-top	persistent mascot	ambient availability
Panel	548 x 560, screen-anchored	mascot + compact chat	quick asks
Full	up to 1060 x 680, centered	complete app UI	work sessions
Fullscreen	entire display, edge to edge	complete app UI, Esc exits	focused immersion

Table 19-1 · The four screen modes and their contracts

The theme architecture's rule is that a theme is complete or it is broken: every surface either consumes tokens or is a listed defect. That rule extends to the settings surface, the approval panel, and the weekly digest, and it is enforced by the polish checklist rather than by memory. The payoff is a product that feels like one object rather than a collage, which is precisely the difference the owner flagged between a prototype and something that seems real.

Theme token architecture

```
:root { --quip-ba: R G B; --quip-text: R G B; --quip-ok: R G B;
        --quip-warn: R G B; --quip-bad: R G B; --chrome-ba: R G B;
        --chrome-text: R G B; --chrome-brand: R G B; ... }
[data-theme='violet'] { ... } /* 10 themes. pre-paint applied */
/* rule: components consume tokens: literals are defects */
```

The architecture's rule is enforced by review and by a scan task in the backlog: a script lists remaining color literals in components, each is either tokenized or documented as sprite art, and the scan joins CI so the count cannot silently grow. The audit's list, status pills, badges, welcome cards, modal gradients, toggle knobs, the weekly digest's whites, and one progression bar wearing Pix's colors, is the seed inventory, and clearing it is what makes the ten themes complete rather than approximate.

Mode transitions and persistence

Mode switching is renderer-optimistic and main-authoritative: the UI changes instantly to avoid perceived lag while the main process resizes and broadcasts the authoritative mode, and disagreement self-heals because the broadcast wins. Geometry persists across boots for each mode, the spatial brain

clamps into the current display, and fullscreen's Esc route is tested alongside its entry, because an immersive mode without a reliable exit is a trap, not a feature. The audit's near-duplicate full and fullscreen layouts are consolidated into one parameterized layout so a fix to one is a fix to both, which is the single-pipeline rule applied to JSX.

Branding placement rules

Branding is quiet and consistent: the logo anchors the chatbox's top-left, theme-aware, sized for the mode; the desktop shortcut icon and tray icon carry the same mark; the companion sprite itself is the deepest brand and is never overlaid with chrome. The rules keep the product feeling like one object across modes and themes, and they are written down because branding by improvisation is how an app ends up with five shades of its own accent color.

Geometry contracts per mode

Mode	Size	Anchor	Always-on-top	Resizable
Companion	132 x 176	bottom-right clamp	yes (screen-saver level)	no
Panel	548 x 560	right-edge anchored	yes	no
Full	up to 1060 x 680	centered	no	yes (min 760 x 520)
Fullscreen	display bounds	edge to edge	no	no (Esc exits)

Table 19-2 · Window geometry contracts enforced by the mode setter

The contracts are enforced in one function, the mode setter, and the geometry table doubles as its test fixture: each mode's size, anchor, and z-order are asserted after switching. Persistence stores per-mode geometry; the spatial brain clamps into the current display on boot and on display-metrics changes; and the always-on-top difference between presence modes and work modes is deliberate, the companion floats, the workspace does not.

The first sixty seconds

The first-run experience is written as a script the startup tests follow: window appears with the companion before any heavy work; the scan overlay shows real stage progress with a skippable affordance; the key setup surfaces in-app with the free-key link; the first task suggestion is one tap from the welcome card. Every step has an honest alternative, no key means the calm no-key message, skip means the scan finishes in background, and nothing in the script requires the terminal. The first sixty seconds are the product's argument for itself, and they are specified with the same rigor as a task plan.

20. Settings, Permissions UX and Proactive Behavior

20. Settings, Permissions UX and Proactive Behavior

Settings is the product's cockpit, organized into eight tabs covering the AI brain, appearance, general, desktop, device, memory, communication profile, and progression. Every control is verified live: provider keys save and probe immediately, the Doctor runs a real connectivity report, voice has a test button, fetch-updates performs a stash-protected pull and reports honestly whether a restart is needed, the desktop shortcut is created through a real Windows shell operation, and the quit path is the only quit path. The audit found no decorative buttons in Settings, which is the correct baseline; the missing piece is the permission-mode switch, fully built in engine and IPC but absent from the interface, and it joins the settings surface as a scheduled task with a plain-language explanation of what each mode auto-approves.

Proactive behavior is deliberately restrained. Check-ins are a user-controlled toggle, suggestions derive from real signals, repeated task shapes, connection events, time-based patterns, and every proactive card is dismissible with the dismissal remembered. Notifications follow the same contract: a companion that interrupts constantly trains its user to close it, so the bar for initiation is that the signal is actionable now, not merely interesting. The honest-empty principle applies here too: when there is nothing worth surfacing, the companion says nothing, and the absence of noise is treated as a feature with tests rather than as empty space.

Error UX completes the trust loop. Failures render in context, a banner for chat errors with a kind-specific next step, per-message error bubbles with a working retry, task failures as plain-language reports with the evidence trail. The onboarding path, first-launch scan, key setup, first task suggestion, is being rebuilt around the window-first startup fix so the very first minutes demonstrate the product's defining trait: it shows up fast, it does what it says, and it tells the truth about what happened.

The permission card, specified

The scheduled permissions card lives in Settings and does four things: shows the current mode in plain language, switches modes with one tap, lists remembered grants with revocation, and links the approval panel's behavior to the mode so the user can predict what will be asked. The copy is specific: Safe mode asks before anything that changes anything; Standard runs read-mostly and app-opening work without asking but still asks before sends, deletes, and purchases; the trust-named mode still refuses the irreversible few unless explicitly allowlisted. The card is the product surface of Chapter 12's machinery, and its absence today is the gap between having a permission system and the user having one.

Proactive cards and the noise budget

Proactivity runs on a noise budget: at most one proactive card at a time, a cooldown after dismissal, and a per-shape mute, so a suggestion the user always dismisses stops coming. Cards derive only from real signals, a repeated goal shape that learned a procedure, a connection event that changed capability, a battery state that matters once the telemetry is real, and each card states its evidence in one line. Check-ins are a toggle with an honest description, and the weekly digest is data from the execution log, never from a template's imagination. The goal is a companion that speaks when it has something to say and is visibly comfortable with silence.

Surface	Real data source	Honest empty state
Weekly digest	execution log + connection journal	no-activity copy, zero fabricated numbers
Proactive cards	procedure store, capability signals	no card rendered
Doctor	live probes with per-provider trail	explicit not-yet-probed state
Device tab	device index + rescan diff	scan-in-progress progress
Memory tab	memory store with scores	nothing-learned-yet copy

Table 20-1 · Every data surface and its honest empty state

Settings information architecture

Eight tabs, ordered by the frequency of need: AI Brain first because keys and voice live there; Appearance next because themes are the most-loved surface; General for companion choice; Desktop for presence, updates, shortcut, and quit; Device for the machine's map; Memory for what Quip knows; Communication DNA for the relationship profile; Progression for the long game. The architecture's rule is that every tab's data is real or honestly empty, the Table 20-1 contract, and every control calls a live handler, the audit's button-by-button verification having found no decorative exceptions. New settings join the tab whose question they answer, never a new tab, because tabs are a budget too.

The quit path's copy

The quit control's copy is a mini-contract: the button says Quit Quip completely, the confirmation says the companion will leave the screen and can be brought back from the desktop shortcut, and the modal's cancel is as large as the confirm. The copy exists because the close-button-already-means-hide semantics can confuse, and the one moment a user truly wants to leave is the one moment the product must be perfectly clear. The audit verified the path's mechanics; the copy task gives the mechanics their voice.

21. Performance and Resource Efficiency

21. Performance and Resource Efficiency

Performance is specified as budgets with owners, not as adjectives. Startup owns the first impression: the target is seconds warm because the window is created before the bootstrap completes and the device index loads from cache; first run is bounded by a visible scan with real progress rather than a frozen taskbar. Interaction owns attention: companion tap-to-panel is frame-fast because the renderer toggles optimistically while the main process resizes; chat first-byte is bounded by provider timeouts and failover. Execution owns trust: every plan step carries a deadline, and the deterministic path now gains the same deadline enforcement as the agent path so no implementation detail can hang a task.

Budget	Target	Enforcement	Measured by
Warm start to visible companion	< 3 s	window-before-bootstrap, cached index	launch log stage timestamps
First run to visible companion	< 15 s	window first, scan streamed live	launch log + scan overlay
Device index lookup	< 50 ms	in-memory index	lookup timing in debug channel
Deterministic task (e.g. open app)	< 1 s end to end	fast path + observation verify	execution log durations
Chat first byte	< 10 s worst chain	provider timeouts + failover	router trail
Idle CPU	near zero	event-driven loops, no polling renders	process sampling
Persistent store growth	bounded per Table 14-1	caps, rings, decay	store size assertions in tests

Table 21-1 · Performance budgets and their enforcement mechanisms

The efficiency audit also names its instruments. The launch log already records stage timestamps; the execution log records per-step durations; the router records tokens and latency per provider. The remaining gap is a small periodic snapshot, process memory, store sizes, ring occupancies, written into the same diagnostics surface the Doctor reads, so regressions surface as trends rather than as user complaints. Measurement precedes optimization in this codebase: every optimization task in the backlog names the metric that will prove it worked.

Startup, measured and budgeted

The startup budget is enforced by the launch log's stage timestamps, which the hardening work made both complete and visible. The budget's shape matters more than its number: window-visible is the user's clock, and it must not wait for network, scan, or model health. Warm start should be dominated by process creation, not by work that could have been cached; first run's only honest excuse is the scan, and the scan must therefore be visible, paced, and skippable-for-the-moment. The backlog's startup tasks each name the timestamp they move, which keeps optimization from becoming folklore about what probably made it faster.

Startup stage	Warm budget	First-run budget	Measured by
Process + single-instance	0.3 s	0.3 s	launch log t0
Window visible (companion)	1.5 s	2.5 s	launch log t1
Device index loaded	0.3 s	background	index timing
Bootstrap complete	1.0 s	bounded by visible scan	stage events
Health probes (background)	n/a	8 s parallel cap	probe log

Table 21-2 · Startup stage budgets

Idle discipline

Idle CPU is a presence tax, and the architecture keeps it near zero by refusing polling wherever an event exists: display changes are events, window closes are events, provider health is on-demand, and the self-heal interval is the only deliberate heartbeat, at fifteen seconds of trivial work. The renderer's animation loop pauses when the window is hidden or minimized; the auto-scroll helper's timer is replaced by event-driven scrolling; and the diagnostics snapshot, once added, runs at a minutes cadence and writes only on change. Idle discipline is also a battery promise, which becomes real the day the battery telemetry does.

The measurement plan

Every budget in Table 21-1 has a measurement path named before optimization begins: launch stages from the launch log, lookups from a debug timing channel, task durations from the execution log, first-byte from the router trail, idle CPU from periodic sampling, store sizes from the diagnostics snapshot. The measurement plan's deliverable is a single diagnostics read-out the Doctor can render, which turns performance from an opinion into a dashboard. Optimization tasks in the backlog are accepted only with their metric named and their before-value recorded, because unverifiable speedups are indistinguishable from placebo.

Token ledger by task class

Task class	Token expectation	Levers when over budget
Deterministic task	0	n/a; investigate why fast path missed
Simple chat	compact prompt only	trim gist slice; tighten identity block
Agent plan	decomposition + decision points	cache tool schemas; narrow tool set
Screen vision	one capture + floored question	crop region; reuse capture; lower retry
Research	reads compressed to relevant spans	better extraction; earlier stopping

Table 21-3 · Token expectations per task class with their levers

Instrumentation points spec

Instrument	Writes	Read by
Launch log	stage timestamps + failure reasons	startup budget checks; troubleshooting
Execution log	per-step records with evidence	digest; post-mortems; fast-path ratio
Router trail	per-request provider attempts	Doctor; token ledger
Observation ledger	captures, vision calls, cache hits	cost review; dedupe tuning
Diagnostics snapshot	memory, store sizes, ring occupancy	Doctor trend; regression watch

Table 21-4 · Instrumentation points and their consumers

The spec's discipline is that each instrument has a consumer, an instrument nobody reads is logging theater. The launch log's consumer is the startup budget test; the execution log's is the digest and this chapter's ratios; the router trail's is the Doctor; the observation ledger's is the cost review; the snapshot's is the trend watch that catches regressions before users do. Together they are the measurement plan made physical, and the backlog's efficiency tasks accept no optimization without naming its instrument.

22. Caching and Indexing Strategy

22. Caching and Indexing Strategy

Caching is the system's memory of the world, and its rules are uniform: every cache has a key, a freshness rule, an invalidation trigger, and a size bound. The device index caches the machine's stable shape with a day-scale time-to-live plus diff-on-boot; the application discovery cache refreshes on the same rhythm; window geometry persists because displays rarely move; provider model lists cache with a health check before reuse; connection outcomes cache into a journal ring that lets the router prefer recently healthy transports. Nothing is cached without an invalidation story, because a stale map is worse than no map: it produces confident wrong actions.

Cache / index	Freshness	Invalidation	Bound
Device index	24 h TTL	diff on boot; manual rescan	size-capped JSON
App discovery index	24 h TTL	diff on boot	size-capped JSON
Window geometry	per move	display-metrics-changed event	one small file
Provider model lists	session	health check failure	in-memory
Observation captures	within task	task end / screen-change hint	ring of 8 files
Connection journal	rolling	ring overwrite	80 entries
Learned procedures	until edited	user edit/delete; trust score	KB-scale records

Table 22-1 · Cache hierarchy with freshness, invalidation, bounds

Indexing strategy follows the same derivation-over-capture rule as storage. Indexes hold answers, not evidence: the device index holds launch identifiers and default-handler claims, not registry dumps; the file discovery index holds paths and metadata, not contents; the knowledge graph holds entities and the links that mattered, capped and decayed. When a cache misses, the miss is an opportunity to learn, the refreshed entry lands with its hash and timestamp, and the next question of that shape is answered from memory. The architecture's speed is not an accident of fast hardware; it is the compounding return on refusing to ask the same question twice.

Invalidation is the hard half

Every cache's invalidation is specified with its key, because an invalid cache is confidently wrong. The device index invalidates on display-metrics events immediately and on a day scale otherwise; the app discovery index shares the rhythm; provider model lists invalidate on any health-check contradiction; the observation cache invalidates on the screen-change hint or task end; window geometry persists but clamps

to the live display on boot. The journal rings need no invalidation because they are append-only truth with a window. What the table in Chapter 22 encodes is a rule the backlog turns into tests: every new cache lands with its invalidation test the same day it lands with its read path.

Cache cost accounting

Caches have a cost side, and the accounting is part of the design: size on disk, freshness work per boot, and staleness risk are all named per entry in the diagnostics snapshot. The diff-only rescans make freshness work cheap, which is what lets the day-scale TTL be generous; the ring buffers make disk cost constant; and staleness risk is bounded by the invalidation triggers being events rather than hope. The architecture's general stance is that a cache is a hypothesis about the future, and the diagnostics make every hypothesis's accuracy observable, which is how the system learns which of its hypotheses deserve to persist.

Hashes and cheap freshness

Freshness is cheap when it is hash-based: device records, app-index entries, and file metadata all carry content hashes, so a rescan is a hash comparison, not a semantic reconciliation. The same trick extends to observation: a screen-region hash detects change without understanding content, letting the verifier wait-for-change cheaply. Hash discipline also bounds write amplification, unchanged records are never rewritten, which matters for the JSON stores on slow disks. The pattern's rule is that any cache whose world can be hashed should be hashed, and the backlog applies it wherever a rescan or recheck currently does more work than a comparison.

The observation cache within a task

Within one task, captures are content-addressed: the first capture establishes the screen's hash, later checks reuse it until something hints the screen changed, an input was synthesized, a navigation completed, a window event fired. The cache's hit rate is visible in the observation ledger, and its correctness is protected by the hint rule, when in doubt, recapture, because a stale screen costs more than a fresh one. This is the rare cache whose miss path is deliberately cheap, which is what makes it safe.

23. Security, Privacy and Trust Boundaries

23. Security, Privacy and Trust Boundaries

The security model is a small set of boundaries held strictly. The renderer runs with context isolation and no Node integration, speaking only through the typed preload bridge; the main process owns keys, filesystem, and shell. API keys persist in the user-data environment file, are masked in every log, and never cross the bridge. Tool execution is deny-listed at the registry, risk-classed at the contracts, and gated by permission modes with an approval surface that names the exact step, target, and consequence. The audit confirmed these boundaries hold and identified the soft spots to watch: the tool surface's shell-reaching primitives remain the highest-value audit target as the capability set grows, and any future move of the renderer into a stricter sandbox must preserve the drag and geometry channels.

Privacy is a local-first contract. Memory, indexes, and logs live in the user's own data directory; conversation archives live in the renderer's storage and are user-clearable; telemetry does not exist, and the network egress list is short and legible, model providers the user configured, voice engines, and web destinations the user's tasks name. The local-versus-cloud boundary is drawn at the capability level: deterministic control and environment knowledge are local forever; model intelligence is cloud by default with Ollama as the offline escape hatch; vision calls leave the machine only when the user's task requires sight and their chosen provider supports it.

Trust is also a product decision about failure. The system never fakes success, never claims an unverified action, and never hides a failure inside an empty catch. Where the audit found silent swallows, and it found several dozen, the roadmap converts each into either an honest report or a documented, justified best-effort with a comment explaining why silence is acceptable there. The security chapter's lasting rule is that every future capability must name its boundary crossing, what leaves the machine, what the user is asked, and what the failure story is, before it ships, and the review checklist in the backlog enforces exactly that.

Threat model, plainly

The threats worth engineering against are concrete: a hostile or compromised renderer attempting registry-and-shell reach through IPC; a malicious prompt or web page steering the agent into destructive actions; a network attacker between the user and providers; and the app itself quietly leaking, keys into logs, transcripts into prompts, screens into storage beyond their ring. Each threat maps to a boundary: typed IPC with no shell in the renderer's reach; risk classes plus approval plus allowlists; TLS with transport selection and no silent proxy override; and the storage and masking rules the audit verified. The model's honesty is that it does not claim to stop a fully compromised OS, only to make the companion a hard target and a loud one.

The secrets ledger

Secret / sensitive item	Stored where	Reaches renderer?	In logs?
Provider API keys	userData/.env, masked at load	never	masked only
Conversation transcripts	renderer localStorage archive	origin	never
Memory facts	userData JSON, capped	summaries only via IPC	names masked optionally
Screen captures	ring of 8, pruned	as file paths for vision	paths only, pruned
Permission grants	userData JSON	shape list only	shape ids
Window geometry	one small file	n/a	n/a

Table 23-1 · The secrets and sensitivity ledger

Privacy as product copy

Privacy is also what the product says about itself, and the copy exists and is true: keys stay local, memory is viewable and clearable, no telemetry exists, egress is the user's configured providers plus the destinations their tasks name. The Settings UI shows the ledger rather than asserting it, and the Doctor can list the egress paths it knows about. When a future capability changes the ledger, the copy changes the same release, because a privacy statement that lags the code is a lie with a delay.

The security review checklist

- **New tool?** Name its risk class, its deny-list status, its deadline, and its verification before merge.
- **New IPC channel?** Declared, typed, handler-guarded, preload-exposed, renderer-used, or it does not merge.
- **New network egress?** Named in the privacy copy and the Doctor's egress list the same release.
- **New persistence?** Bound, decayed, or ringed, with the growth regression test in the same commit.
- **New key or secret?** Main-process only, masked in logs, never crossing the bridge, in the ledger.
- **New approval surface?** Shows step, target, consequence, and lifetime choice; denial is consumed by the planner.

The checklist is the chapter compressed into review comments, and the backlog enforces it by requiring its boxes in the definition of done for every task that touches the named surfaces. It exists because security failures are almost never exotic; they are ordinary features missing one line of the checklist, and the checklist is cheaper than the incident.

24. Reliability Engineering and Lifecycle

24. Reliability Engineering and Lifecycle

Reliability is engineered at the lifecycle seams, because that is where desktop applications actually fail. Startup: the single-instance lock routes second launches to the existing window; bootstrap is fail-soft with per-stage catches; the window exists before the heavy work so a slow scan never reads as a dead app. Runtime: renderer crashes are recovered with reload-then-recreate; an unresponsive renderer is force-crashed and rebuilt; a self-heal interval re-shows a hidden companion whose window was lost. Shutdown: the close button hides, the quit path flushes memory and marks the intent so interceptors stand down, and the tray provides the same two verbs, show and quit, outside the renderer entirely.

The lifecycle test regime in the backlog makes these properties repeatable: repeated launch, close-to-companion, reopen cycles; quit-path verification that the process actually exits; second-instance focusing; crash-recovery simulation by killing the renderer; boot with no keys, bad keys, and offline network; and a clean-start test that wipes the user-data directory to prove the first-run path stays honest after every change. Each cycle asserts both the visible truth, the companion appears, and the logged truth, stage timestamps within budget. The regime exists because lifecycle bugs are the ones users describe as “it just doesn't open,” and those four words cost more trust than any missing feature.

Failure containment follows one rule: optional services fail gracefully, mandatory services fail loudly. Voice failing must not dim the chat; vision failing must return a clear error; a provider chain exhausting must produce the calm no-key message the codebase already renders; a device rescan storm must be single-flighted rather than concurrent. The reliability chapter's definition of done for any feature is that its failure modes are named in the plan, exercised in the tests, and rendered in the UI, and the backlog's verification tasks check precisely that trio for every capability the roadmap adds.

The lifecycle test matrix

Cycle	Assertion	Automated?
Launch (clean user-data)	companion visible; stages within budget; scan real	sandbox + Windows checklist
Launch (warm)	companion visible < 3 s; index loaded from cache	sandbox
Second instance	existing window focused; no second process	sandbox
Close (X)	window returns to companion; process alive	renderer + main test
Quit from Settings	process exits; memory flushed; no stray timers	sandbox
Renderer crash	reload x3 then recreate; companion returns	simulated kill

Cycle	Assertion	Automated?
Unresponsive renderer	force crash + recreate; no zombie	simulated hang
No keys / bad keys / offline	calm no-key message; honest Doctor	sandbox

Table 24-1 · Lifecycle test matrix and where each runs

The matrix's power is that it is regression armor: each row is a test that runs on every build in the sandbox where possible, and the Windows-only rows run on the release checklist with the same assertions. The clean-start row is repeated at every phase gate, because the first-run path is the most easily broken by innocent changes and the most expensive to break in public. The audit's baseline, four hundred twenty-five passing checks, becomes the floor under this matrix, not the ceiling.

Containment in practice

Containment rules are small and absolute. Optional capability failure degrades the feature, never the process. Background work owns its timers and cancels cleanly on shutdown. Concurrent storm-prone operations, rescans, probes, discovery walks, are single-flighted with a last-request-wins policy. Long primitives poll their cancellation token. And every containment rule has a test that tries to violate it, because a containment rule without an adversarial test is a comment. The reliability chapter's promise is that the user's machine stays theirs: the companion earns its persistence on the desktop by being polite in the process table.

The incident playbook

When something breaks in the field, the playbook is: reproduce with the launch or execution log; classify by the lifecycle matrix row it matches; fix at the root with the audit's method; add or extend the test for that row; and record the whole arc in the bugs ledger from discovery to verified fix. The playbook's discipline is that no fix ships without its test, and no incident closes without its ledger entry, which converts field failures into permanent regression armor one issue at a time. The overnight run itself followed this playbook, and its ledger is Appendix D's artifact.

Shutdown correctness

Shutdown is a reliability surface of its own: the quit path sets the quitting flag so window interceptors stand down, flushes the memory store, closes journals cleanly, and exits with no stray timers; the launcher's console window and the tray exit the same path; and a killed process, power loss, crash, leaves stores in recoverable states because every write is atomic-per-file and debounced. The shutdown tests assert process exit and log completeness, because a companion that leaks its own lifecycle into the next boot is failing quietly at the exact moment users notice most.

25. Current Codebase Audit

25. Current Codebase Audit

This chapter summarizes the verified audit; the full evidence trail lives in the repository's CODEBASE_AUDIT.md and is kept synchronized with the bugs ledger. The audit read the electron main process, engine, brains, brain, system, and actions directories, and the complete renderer, cross-checked the IPC registry against handlers and preload exposure, and validated the baseline, build green, 425 tests passing, at commit ae6b1ab. Findings carry file-and-line evidence; anything the sandbox could not execute, Windows-only shell operations, real provider endpoints, is marked unverified rather than assumed.

High-severity findings

ID	Finding	Evidence	Fix strategy
B1	First-run device scan blocks first window; scan animation can never show on first launch	createWindow after await bootstrap (main.ts:2066 to 2174); broadcast hits empty window map (main.ts:2039)	Create window before bootstrap; buffer and flush progress events; scan stays visible and real
B2	System prompt built outside CHAT_SEND try; a brain throw yields unhandled rejection, not styled error	buildSystemPrompt call at main.ts:964 precedes guarded block	Wrap prompt assembly; emit CHAT_ERROR with honest reason
B3	Deterministic orchestrator path has no deadline; scrapes lack AbortSignal; hung fetch stalls task	orchestrator executeWithRetry (503-524); browser-automation fetch (228-231); legacy-tools (116)	Per-step deadline wrapper; AbortSignal on scrapes; honest timeout failure
B4	Knowledge graph and timeline grow unbounded with synchronous whole-file writes per event	knowledge-graph.ts:91-96; timeline-brain.ts:47	Caps + decay + debounced writes aligned with other stores
B5	Weekly digest fabricates statistics, violating the never-fake-success rule	WeeklyReflection.tsx:14-24 hardcoded 14 tasks / 3 sessions / 2 hours	Render real digest from weekly summary API; honest empty state; wire feedback
B6	Scan overlay can mark first-run done on a timer regardless of real bootstrap state	ScanOverlay.tsx:36-39 fallback	Drive completion from real bootstrap-done event with generous timeout

Table 25-1 · High-severity findings with evidence and fix strategy

Medium and low findings

The medium tier contains the architecture's coherence debts: the legacy planning trio of task-brain, tool-executor, and capability-registry, about 1,200 lines loaded but unreachable except through one unused import; three overlapping risk tables that should be one; two YouTube search implementations; a permission-mode backend with no settings UI; the battery signal hardcoded to a constant, silently disabling low-power awareness; and the boot health probe's serial, proxy-blind churn. The low tier collects the polish facts: roughly two dozen components or files with hardcoded colors bypassing theme tokens; two companions ignoring cursor tracking; a progression bar wearing one companion's colors for all; dead hooks and dead CSS; stale comments; unguarded settings handlers; and a scroll helper polling on a timer. None of these is individually dramatic; together they are the difference between a codebase that works and a codebase that reads like a product.

The audit also recorded what is right, because a fix list without a strengths list misleads the roadmap. The provider chain with circuit breaking and honest trails is production-grade. The execution log, ring-bounded and debounced, is exactly the pattern the two unbounded stores should copy. The single window factory with mode geometry is clean. The test suite's 425 checks, including a static IPC contract checker, give the refactor work a safety net most projects of this size lack. And the close-to-tray, quit-only-from-settings semantics are already correct end to end, which the audit verified rather than assumed.

Medium-tier findings detail

ID	Finding	Evidence	Disposition
B7	Permission modes: full backend, zero UI	engine modes live; IPC present; no renderer call site	P1 task: settings card + grants store
B8	Battery telemetry hardcoded; low-power features dead	environment-brain snapshot constant	real read with 60 s cache; honest unsupported
B9a	Legacy planning trio loaded but unreachable	sole lifeline: unused runTask import (main.ts:89)	remove import; delete trio; tests re-pointed
B9b	Three overlapping risk tables	permission-system / permission-modes / contracts	single source in action contracts
B9c	Duplicate YouTube search implementations	tool-executor (dies with trio) + browser-automation	resolved by B9a deletion
B9d	Off-registry IPC literals	set-companion, permission-mode trio, approval-resolve, swarm raw	move into shared.ts registry
B9e	Dead renderer surface	useKeyboard, useDeviceProfile, useSpatialLayout; dead CSS classes	delete; wire Esc where it belongs
B10	Boot probe: serial, proxy-blind Node fetch	main.ts:2195-2198	parallel, 8 s cap, router transport

Table 25-2 · Medium-tier findings and dispositions

Low-tier polish inventory

The polish tier is enumerated so it cannot be dismissed as vibes. Theme leaks: status colors in `TopBar`, `ChatMessage` badges, `ChatWelcome` cards, `ConfirmModal`, `ActionApprovalPanel`; toggle knobs; the `WeeklyReflection` component ignoring themes wholesale; the progression bar's hardcoded duo. Companion consistency: two bodies ignoring eye offset; art-side literals outside the config module; one stale name comment. Robustness: four unguarded settings handlers; a dead ternary; a scroll helper on a timer; a pose without a clear timer. Housekeeping: unused imports in main; one dead IPC channel; two stale theme keys; dead CSS classes. Each item is a small, mechanically verifiable fix, and the backlog carries them as a batch with a shared verification, the token-scan and import-graph scripts in CI.

What the audit refused to conclude

Three areas are marked unverified rather than guessed: Windows-only shell operations, including shortcut creation and screen capture, are code-reviewed but not executed in this Linux sandbox; live provider behavior is tested through contract and mock, not against paid endpoints; and long-horizon storage growth is bounded by design and tests rather than by a month-long soak. These are recorded as such in the capability map, each with the environment it needs for true verification, because a report that pretends to know more than it measured is the same failure the product is forbidden to make.

The tool inventory digest

The live tool registry is the audit's proudest inventory: forty-plus registered capabilities spanning applications, input, browser, filesystem, screen vision, system control, knowledge, and the agent-reach channels, each with deny-list screening and risk classification. The digest's purpose is orientation, not duplication: the registry file remains the source of truth, the capability map records each entry's status, and the audit's contribution was verifying that the registry's entries are reachable from the startup or IPC graph, which the import-graph check now guards permanently. The registry's growth rule remains the composition rule from Chapter 5, primitives over features.

Dead-code deletion as an architecture act

Deleting the legacy trio is not housekeeping; it is the architecture asserting itself. Three files that replicate the parse-plan-execute pipeline, kept alive by one unused import, are exactly how systems grow contradictory: a future contributor finds the old path, assumes it is the real one, and extends the wrong thing. Deletion with tests green is the proof that the live pipeline is the pipeline, and the import-graph check keeps the proof true. The audit's disposition, delete, not quarantine, is deliberate: quarantines are attics, and attics fill.

Findings-to-tasks traceability

Every finding in this chapter maps to at least one backlog task with an identifier, and the bugs ledger links the task to the test that closed it: B1 to the window-first bootstrap task and its boot assertions; B2 to the prompt-guard task and its throwing-brain test; B3 to the deadline task and its hanging-fetch test; B4 to the

store-bound tasks and their soak tests; B5 to the digest rewrite and its zero-fabrication check; B6 to the overlay's real-done wiring; B7 to the settings card; B8 to the battery read; B9's deletions to the import-graph gate; B10 to the probe rewrite. The traceability is the audit's product: findings that cannot become tasks are opinions, and opinions do not ship.

The audit method, so it can be repeated

The method is four passes, repeatable at any future phase gate. Read: every module in the execution spine, entry to exit, with file-and-line notes. Cross-check: registry against handlers, handlers against preload, preload against renderer usage. Execute: build, full test suite, clean-start boot, lifecycle rows the environment permits. Reconcile: every claim against evidence, marking unverified what cannot run here. The passes produce this chapter's shape every time, which means the next audit measures drift from this one rather than starting from zero, and drift measurement is how architecture stays real.

26. Skales Integration Strategy

26. Skales Integration Strategy

Skales entered this project through two doors, and the strategy keeps them distinct. The first door is companionship: the Skales gecko, originally from the Skales app's pet roster, was redrawn in Quip's own pixel-sprite language and ships as one of the six companions, with credit maintained in the code. The second door is engineering: Skales-flavored capability work in this repository, document tools, weather, system information, and the agent-reach channel family, was implemented as Quip-owned code registered in Quip's own tool registry, not as vendored modules. The integration rule that follows: inspiration and identity may be shared; licenses and boundaries may not be crossed, and where any external code's license forbids direct reuse, Quip implements the technically equivalent approach and records the decision.

The convergence path treats Skales-style computer use as a capability source, not a second engine. Anything the integration could contribute, screen-reading heuristics, browser recipes, input strategies, must enter through the registry as primitives with contracts and verification, reviewable under Quip's permission model; nothing enters as a parallel executor that bypasses the action engine's gates. Dependency health is checked like any other channel: probes, fallbacks, and honest failure, and the capability map records each integration as live, partial, or blocked with the reason. This keeps the companionship joy of the Skales family while ensuring the engineering remains one system with one set of rules.

What Skales contributed, precisely

Contribution	Form	License posture	Integration point
Skales the gecko companion	redrawn original sprite art	identity with credit	companion roster entry
Document tools capability	Quip-owned implementation	equivalent approach	tool registry entries
Weather and system info	Quip-owned implementation	equivalent approach	tool registry entries
Agent-reach channels	Quip-owned implementations	documented interfaces	tool registry entries

Table 26-1 · Skales-family contributions and their posture

The table is the strategy in one view: contributions enter as Quip-owned, registry-integrated capabilities or as credited identity, never as vendored code paths with unknown terms. The audit verified the live wiring of the registry entries and the roster entry; the licensing column records the posture decision rather than a legal opinion, and any future change, a desire to vendor genuinely, for instance, restarts the review rather than drifting.

Computer-use convergence

The interesting convergence is engineering: computer-use systems of the Skales variety and Quip's own controller are after the same prize, described targets, observed results, verified outcomes. Quip's path is to absorb the ideas, description-based targeting, state-diff verification, recovery ladders, through its own contracts so everything inherits permissions and logging. Where an external system demonstrates a capability Quip lacks, the backlog task is to implement it as a registry entry with a test, and the capability map records the lineage honestly. Convergence by education, not by adoption, keeps one engine, one risk table, and one trust story.

Attribution and the companion's name

Skales the gecko appears in the roster with credit preserved in the config file's comment trail, because identity attribution is cheap and right. The redraw is Quip's own pixel-sprite language rather than an asset copy, which keeps the visual system uniform across all six companions while honoring the character's origin. The same posture governs any future adopted identity: credit in code, redraw or equivalent implementation in fact, license respected by default, and the decision recorded in this chapter's table so the posture is auditable rather than folklore.

Integration health in the Doctor

Integration health is a Doctor row like any other: each channel family, docs, weather, system info, agent-reach, reports its probe status, last success, and failure reason if any. Unhealthy integrations are skipped by the planner rather than retried into requests, and their recovery is passive, probe-driven, so the system heals without user intervention while remaining honest about what is degraded. The health rows make the difference between an integration that exists and one the user can rely on, and they are why the

capability map's live column is a query, not a claim.

27. Testing Strategy and Observability

27. Testing Strategy and Observability

The test suite, 425 checks across twenty-plus files at baseline, is the project's safety net and its specification of record for behavior. Its current strengths are contract tests, IPC registry consistency, parser behavior, provider chain mechanics, execution-log bounds, launcher correctness, and the fullscreen and launcher rounds added with their features. Its gaps are the classic ones of a fast-moving product: renderer components are tested indirectly through data contracts rather than interaction; Windows-only paths are code-reviewed but not executed in the Linux sandbox; and a handful of newly scheduled fixes, window-first bootstrap, prompt-guard, store bounds, will land with tests written alongside rather than after.

The target pyramid keeps contract tests as the wide base, because the IPC boundary is where this app's regressions actually bite: the static checker that validates shared-registry, handler, and preload alignment is being extended with a renderer-usage diff so a channel that loses its consumer or its handler fails CI rather than the user. The middle layer adds lifecycle and execution tests, boot with empty or hostile user-data, repeated open-close-reopen, plan execution with failing verifications, cancellation mid-step. The narrow top reserves end-to-end GUI runs for the Windows checklist executed at release time, since the sandbox cannot render a display. Every backlog fix names its test in the task text, and the bugs ledger links the test that proved the fix.

Observability is the runtime half of testing. The launch log records stage timestamps and, after the launcher hardening, any failure with its real reason; the execution log records plan trails for post-mortems; the Doctor screen turns provider health into a readable report; the router's per-request trail names every provider tried and why each was skipped. The planned diagnostics snapshot adds process memory and store sizes to the same surface. Together they make the system explainable after the fact, which is what lets a small team operate a product this ambitious without heroics.

Test file map and what each layer owns

Layer	Representative coverage	Gap being closed
Contract	IPC registry alignment; parser; provider chain; launcher	renderer-usage diff joins the checker
Engine	understanding, planner, action engine, recovery, log bounds	deadline tests for deterministic path

Layer	Representative coverage	Gap being closed
Stores	memory cap, ring buffers, execution log	graph/timeline bound soak tests
UI data	theme catalog, companion roster, mode registry	token-literal scan as CI gate
Lifecycle	boot, keys, updates, fullscreen rounds	clean-start matrix automation

Table 27-1 · Test layers, coverage, and the gaps being closed

Writing tests as specification

The suite's most valuable habit is that tests are written as behavior specifications, not as echoes of implementation. The launcher round tests the promise the README makes to users; the fullscreen round tests the mode contract end to end; the store tests assert bounds rather than implementation details. The backlog continues the habit: every fix task carries its test text, and several audit fixes are specified primarily by the test that will prove them. When the code and the specification disagree, the test arbitrates, which is what keeps a fast-moving product honest at speed.

Observability deliverables

Three artifacts make the system explainable: the launch log with stage timestamps and failure reasons, the execution log with per-step trails, and the Doctor's provider report. The planned diagnostics snapshot adds process memory, store sizes, and ring occupancies on a minutes cadence. Together they answer the four questions any incident raises, what ran, what did it see, what did it claim, and what did it actually observe, without attaching a debugger. Observability is also a politeness feature: when the user asks what happened, the answer is a read-out, not an investigation.

CI pipeline spec

```
on push:
  1 npm ci                # lockfile-exact deps
  2 npm run build         # electron tsc + vite
  3 npm test              # 425+ checks incl. store bounds
  4 node scripts/inc-audit.cis # registry/handler/preload/usage
  5 node scripts/import-graph.cis # reachability + allowlist
  6 node scripts/token-scan.cis # theme literal count (<= baseline)
  7 clean-start smoke     # empty user-data boot assertions
```

The pipeline's steps map one-to-one to this document's contracts: build and tests to behavior, the IPC audit to the boundary, the import graph to single-pipeline ownership, the token scan to theme completeness, and the clean-start smoke to the lifecycle matrix's first row. Every gate is a script that already exists or is specified in the backlog with its failure message, and the gates are intentionally cheap, minutes, not hours, so they run on every push rather than on ceremony.

Test data and fixtures discipline

Tests that touch stores use fixtures built by the same schemas the code uses, never hand-crafted JSON that drifts; tests that touch time use injectable clocks; tests that touch the screen use captured fixtures with known hashes; tests that touch providers use the recorded trail format. The discipline keeps the suite fast and honest, and it means a fixture's schema change breaks the tests that should break, the property that makes the suite a specification instead of a museum.

28. Packaging, Updates and Migration

28. Packaging, Updates and Migration

Today's packaging is honest: a launcher script that checks Node, installs dependencies idempotently, builds only when sources changed, launches, logs every step to a file, and pops a real Windows message box with the true reason when any step fails. The audit verified this contract and its tests. The target removes the remaining friction in two steps: first, a bundled launcher that carries the runtime so the Node prerequisite disappears for non-developer users, and second, a proper installer that registers the app, creates the shortcut, and preserves user data across updates. Neither step changes the code's contract with the launcher; both shrink the distance between clone and companion.

Fetch Updates already implements the safe pattern: stash local changes, pull with fast-forward only, restore the stash, and report honestly when conflicts require a human, with a needs-restart flag the settings surface honors. The migration discipline extends that honesty to data: every store schema change ships with a versioned reader that upgrades in place, never destructively, and the clean-start test proves a fresh install and an upgraded install both boot. The packaging chapter's rule is that updates are a feature with failure modes like any other, so they get the same verification trail, logs, honest reports, and tests, as the rest of the system.

The launcher contract, line by line

The launcher's promises are explicit and tested: Node presence is checked with a human message; dependency installation is idempotent and always safe to re-run; the build step runs only when sources are newer than the build stamp, making warm starts seconds; every step appends to a launch log a user can open; and any failure pops a Windows message box with the actual reason, never an invisible death. The audit verified this contract, and the hardening work that produced it came directly from the user-facing failure this project began with: an app that would not open and said nothing. The contract's rule for the future is that new prerequisites join the checked chain rather than being assumed, because assumptions are where startup bugs live.

Update and migration safety

Fetch Updates implements the safe sequence, stash, fast-forward pull, pop, honest conflict report, and the needs-restart flag is honored by the UI rather than merely returned. Data migrations follow two rules: readers are versioned and upgrade in place, and a migration that must trim does so with a report of what was removed. The clean-start test proves both worlds, fresh install and upgraded install, boot within budget, and the backlog keeps that test at every phase gate. Packaging's future work, a bundled runtime and a real installer, preserves these contracts so the distance between clone and companion shrinks without the safety net shrinking with it.

Distribution step	Today	Target
Runtime	user-installed Node	bundled runtime
Install	clone + launcher script	installer + shortcut
Update	Fetch Updates (stash-protected)	same, plus delta-friendly layout
Data migration	versioned readers	same, tested at gates
First-run	scan visible after window-first fix	onboarding key flow in-app

Table 28-1 · Distribution evolution without contract changes

Installer flow, target state

The target installer's flow is five steps: verify and lay down the bundled runtime; copy the application; create the desktop and start-menu shortcuts with the bold icon; register the uninstaller; and preserve any existing user-data directory untouched. Updates prefer the in-app Fetch Updates path, which keeps its stash-protected semantics, with the installer as the repair and first-install vehicle. The installer changes nothing about the launcher's contract, it replaces the human's part of it, and the clean-start matrix gains an installer row so the first-run experience is tested through the same door users will actually enter.

Release checklist (Windows)

Check	Evidence
Launcher path: double-click to companion	launch log with budgets met
Installer path: install to companion	installer log + first-run script
Fetch Updates round-trip	stash protection verified; restart flag honored
Shortcut icon legibility	manual eyeball at 100% and 150% DPI
Quit path exits process	process absent; memory flushed
Full lifecycle matrix rows 1-8	checkboxes with dates

Table 28-2 · The Windows release checklist rows

29. Future Capabilities

29. Future Capabilities

This chapter extends the capability model into plausible futures, each described as system behavior with its infrastructure requirement, sorted into near, medium, and long horizons. Near-term means the architecture already contains the seams; medium-term means a named subsystem must be built; long-term means a genuine research or platform dependency exists. Nothing here is promised; everything here is priced. The discipline is the same one that governs the present: every capability enters through the registry, carries verification, and respects the efficiency thesis.

Near-term (architecture seams exist)

- **Sentence-streamed voice.** Pipe sentence-level synthesis while text streams; infrastructure: TTS chunking plus renderer audio queue.
- **Voice input, push-to-talk.** Local or provider speech-to-text feeding the same understanding object; infrastructure: capture pipeline and a provider abstraction for ASR.
- **Session history UI.** The archive store already exists; the surface is a list, a search box, and an open action.
- **Permission profiles per task shape.** Extend modes with per-shape remembers, for example auto-allow opening this app; infrastructure: contracts gain a shape id and the store gains grants.
- **richer task progress.** Step list with live verify evidence in the full app; infrastructure: execution log streaming to renderer.

Medium-term (named subsystems to build)

- **Procedure marketplace of one.** Export and import learned procedures between machines; infrastructure: signed procedure bundles plus a review UI.
- **File intelligence.** Content-aware summaries and semantic file search over declared working areas; infrastructure: local embeddings store, capped, with a diff-maintained index.
- **Browser co-pilot mode.** Read the user's active tab through an extension bridge with explicit grants; infrastructure: extension plus permission contract for observation-only mode.
- **Calendar and mail actions.** Compose, schedule, and draft through account-aware destinations; infrastructure: destination adapters with approval-grade contracts.
- **Proactive routines.** User-authored triggers, when I say X, weekdays at Y, run procedure Z; infrastructure: scheduler plus the procedure store.

Long-term (platform or research dependency)

- **Reliable general GUI grounding.** Element-level understanding of arbitrary applications beyond vision models; dependency: OS accessibility APIs and model grounding research.
- **Cross-device presence.** The companion follows the user across laptop and phone with shared memory; dependency: sync infrastructure with a privacy-preserving design.
- **On-device small-model reasoning.** Classification and routing fully offline; dependency: local model runtime quality catching provider quality.
- **Natural-language workflow authoring.** Users describe a routine; the system drafts a procedure for approval; dependency: robust procedure verification at draft time.

The catalog's honesty test is that each item names what must exist before it can exist. That test keeps the roadmap a plan rather than a wish list, and it gives every future feature request a place to land: if it appears here, it has a price; if it does not, the first task is to price it.

Pricing the near-term honestly

The near-term list is short because near-term means the seams exist and the work is surface, not substrate. Sentence-streamed voice needs a synthesis chunker and a renderer audio queue; push-to-talk needs a capture path and an ASR abstraction behind the existing provider pattern; session history needs a list, a search, and an open action over a store that already exists; permission grants need the store and card Chapter 12 specified; the richer task progress view needs the execution log streamed to the renderer it already serves. Each names its files and its tests in the backlog, and none requires a new subsystem, which is exactly what qualifies it as near-term.

Medium-term subsystems and their first milestones

Medium-term items earn their own first milestone definitions so they cannot sprawl. File intelligence's first milestone is semantic search over declared working areas with a capped store and a diff-maintained index, not a knowledge base over the whole disk. Browser co-pilot's first milestone is observation-only reading of the active tab behind an explicit grant, not control. Calendar and mail actions' first milestone is compose-and-draft with approval-grade contracts, never silent sending. Proactive routines' first milestone is user-authored triggers over the procedure store, not autonomous habit invention. Each subsystem inherits the lifecycle, the risk classes, and the verification, which is what makes its risk surface a known shape.

Long-term dependencies, named

The long-term list is honest about dependencies. Reliable general GUI grounding waits on accessibility APIs and model grounding quality beyond today's vision loops, and the architecture's contribution is the observation service those advances will plug into. Cross-device presence waits on a privacy-preserving sync design, and its architecture contribution is compact stores worth syncing. On-device reasoning waits on local model quality, and its contribution is the routing market ready to price a local provider. Natural-language workflow authoring waits on draft-time verification, and its contribution is the procedure schema. The document's stance is to keep the hooks real and the hype zero.

Capability pricing template

Every future capability, whatever its source, is priced with the same template before it enters the roadmap: the behavior in one sentence; the infrastructure it needs; the registries it touches; its risk classes; its verification; its storage and token budgets; and its phase. The template is short on purpose, an afternoon of honest answers, and its refusal to admit prose is what keeps the catalog grounded. The Chapter 29 lists were written with it, and every backlog item sourced from them carries its filled template in the task text.

Deliberately declined ideas

A vision earns credibility with what it declines. Quip does not plan an always-listening ambient microphone, a cloud-synced memory store without a local-first design, an app-store economy inside the companion, autonomous purchasing, or background surveillance of the user's activity beyond task-scoped observation. Each declined idea violates a chapter-2 anti-goal or a chapter-23 boundary, and writing them down prevents their quiet return dressed as a feature. The declined list is revisited only when the architecture that would make them safe exists, which is a higher bar than desire.

Deep-dive: semantic file search

The near-horizon file-intelligence milestone is specified enough to build. Scope: declared working areas only. Index: file chunks embedded once, stored capped, refreshed by the diff discipline on file-change events. Query: natural-language description from the chat, ranked candidates with path and span evidence. Verification: opening the candidate is the ground truth; a wrong candidate teaches the ranker a negative. Budgets: the store caps at a few megabytes per ten thousand chunks; queries are local and instant; embeddings refresh only on change. The milestone's exit is a test that finds a fixture file by description it never contained verbatim, the definition of semantic working.

Deep-dive: observation-only browser co-pilot

The browser co-pilot's first milestone reads without acting: a companion extension surfaces the active tab's readable content and metadata to Quip behind an explicit, revocable grant, observation-only in the permission system's own vocabulary. The grant's scope is the tab, the session, and the read verb; nothing clicks, nothing types; and the grant's indicator lives where the user can see it. The milestone's exit is a research task, “summarize what this page says about X,” answered with a citation to the live tab, zero synthetic input, and the trail showing the grant was honored. Control comes later, behind the same contracts as everything else; reading comes first because reading is where trust is built.

30. Worked Examples: Task Walkthroughs

30. Worked Examples: Task Walkthroughs

These walkthroughs trace real requests through the lifecycle of Chapter 10, naming the stage owners and the cost profile at each step. They are the acceptance stories for the architecture: if these traces match the code's actual behavior, the system is what this document says it is.

Simple: “open VS Code”

Understanding classifies an app-launch goal with target VS Code; the device index supplies the installed launch identifier deterministically; capability discovery selects the launch primitive; the plan is one Medium step with the expectation that a window whose title matches appears; execution launches and the verifier observes the window; completion reports with a trust line naming why the installed app was chosen. Cost: zero model tokens, under a second end to end. The same request on a machine without VS Code produces discovery through the index, an honest miss, and, only if the user's phrasing implies a web alternative, a web-destination suggestion, never a silent web search.

Medium: “find the newest PDF I downloaded and open it”

Understanding resolves a file-query plus open composition with a temporal filter; the file discovery index answers newest-in-Downloads from metadata without a scan; the plan carries two steps, resolve and open, each with expectations, path exists, then association launch observed; a Dangerous-classification is not triggered because opening is Medium. If two candidates tie on timestamp, the clarification gate asks which, presenting the two names, rather than guessing; the chosen one becomes a learned referent for “the file I meant.” Cost: zero tokens unless clarification is needed; the exchange itself becomes a reusable procedure keyed to the shape “newest file in folder.”

Compound: “open my Quip project, find why the build fails, fix it, run the tests, tell me what happened”

Understanding marks this agent-shaped with a workspace target; the workspace context and file index locate the project; decomposition produces a sub-plan chain: open workspace, run build capturing output, interpret failure with model assistance grounded in the actual error text, apply the edit through file primitives with a diff shown, run the test suite, and report. Each loop through execute-observe-verify lands evidence in the execution log; checkpoints between stages mean a failed build interpretation asks rather than guesses an edit; the edit step is approval-gated by risk class in Safe mode and remembered grants thereafter. The final report is a trail summary: what failed, what was changed, what the tests now say, each claim backed by an observation. Cost: a handful of model calls at genuine decision points, everything else deterministic, and the whole run leaves a procedure skeleton the next similar request can

reuse.

The pattern across all three is the same spine at different depths, and the pattern across their costs is the efficiency thesis in action: tokens appear only where judgment appears, and judgment that recurs becomes procedure, and procedures become free.

Recovery walkthrough: the verification that failed

A walkthrough of failure teaches more than a walkthrough of success. Consider “open the Q3 report and put the totals in an email draft” where the file open succeeds but the page's totals table renders late. The open step verifies by process and window, passes; the read step's structured query returns empty for the region; the verifier marks unclear, recovery re-resolves after a short wait, the second query returns the table; the compose step proceeds; the draft step is approval-gated, allowed, and verified by the draft existing with the numbers present. The trail in the execution log shows every beat: one unclear, one re-resolve, one grant, two verifications. The user sees a short pause and a correct draft; the log shows why. That is the recovery ladder doing its job invisibly.

Cross-application walkthrough: research to file

“Look up the TypeScript 5.5 release notes and save a summary in my notes folder.” Understanding marks this agent-shaped with a web evidence step and a filesystem write; discovery resolves the notes folder from declared working areas; the web step reads the release notes through the reading pipeline with the URL cited; the compression step is model-assisted with the source span grounded; the write creates a dated markdown file, verified by existence and shape; the completion names the source and the path in one trust line. Tokens appear at compression only; the procedure stores the shape for next time; the whole run lands in the digest as one verified trail.

What these walkthroughs commit the architecture to

- Every claim in a completion message maps to an observation in the trail; nothing is narrated from hope.
- Every model call in a walkthrough sits at a judgment point; the mechanical beats are deterministic.
- Every approval appears at a consequence, not at a quota; remembered grants shrink future asks without widening scope.
- Every completed walkthrough leaves either a reusable procedure or a marked reason why not.
- Every failure inside a walkthrough produces the same artifacts as success, a trail, a reason, and a lesson.

Walkthrough: the interruption

One more trace, because interruptions are where companions fail. Mid-plan, the user asks something unrelated, “what's the weather tomorrow?” The engine holds the running plan's state, answers the interruption through the fast path, a knowledge tool with the weather channel, and returns the plan to its

step boundary unchanged; the companion's states express the switch, working to responding and back. The interruption never queues behind the plan's completion, because attention is the product; and it never corrupts the plan, because the plan's state machine is reentrant at step boundaries. The trace's lesson: concurrency of conversation and execution is designed, not hoped for.

Walkthrough: the compound failure

The compound objective from the chapter's opening fails halfway when the build's error output reveals a missing dependency the fix cannot install without a network decision. The trail shows the decomposition, the successful open and build, the interpretation step's grounded read of the error, the recovery ladder's honest end: the install step is approval-gated as a system-changing action, the user is away, the plan parks in `WAITING_FOR_PERMISSION` with a clean summary of everything verified so far. When the user returns, one tap finishes the run, and the digest records it as a single task with a pause, because that is what happened. Failing well is the architecture's signature move, and this is what it looks like in the log.

31. Roadmap: From Prototype to Vision

31. Roadmap: From Prototype to Vision

The roadmap orders the work by the owner's priority: startup and reliability blockers first, then architecture coherence, then core task execution, then computer-use depth, then integrations, then efficiency, then the UX program, then advanced capabilities, then polish and verification. Phases are gates, not dates: a phase exits when its exit criteria verify, and the backlog's task identifiers trace every line item to its phase.

Phase	Focus	Exit criteria (verified)
P0	Startup and reliability blockers	Window-first bootstrap live; prompt-guard and deadlines in; clean-start and lifecycle cycles pass; B1-B3, B6 closed
P1	Architecture coherence	Legacy trio deleted with tests green; single risk table; stores bounded; permission UI ships; weekly digest real; B4-B10 closed
P2	Efficiency and observation	Budgets in Table 21-1 measured; cache table 22-1 enforced; observation service dedupes captures; probe transport fixed
P3	UX and companion program	All theme leaks tokenized; six companions consistent incl. eye tracking; progression colors per companion; session history ships
P4	Capability depth	Screen-vision confidence floors tuned; procedures replay with trust scores; browser co-pilot observation mode behind grants
P5	Polish and verification	Full checklist green across themes and modes; Windows release checklist executed; audit and bugs ledger at zero open high-severity

Table 31-1 · Phased roadmap with verified exit criteria

Two rules govern sequencing. First, no phase begins with open P0 failures, because everything built on a broken foundation inherits the break. Second, every phase ends with artifacts synchronized, audit, bugs ledger, capability map, and this document's relevant chapter updated, so the next session starts from truth rather than archaeology. The roadmap's promise is not a date; it is an order, and the order is chosen so that at every gate the product is the most trustworthy version of itself built so far.

The backlog in brief

The full engineering backlog generated from this document exceeds four hundred tasks across eighteen domains, stored as the repository's BACKLOG.md with dependency tiers, acceptance criteria, and verification methods per task. Its shape follows the phase table: P0's startup and reliability blockers are small and sharp; P1's coherence work is the largest deletion-and-consolidation pass; P2's efficiency tasks are measurement-first; P3's UX program is the theme, companion, and settings sweep; P4 and P5 carry capability depth and final verification. The backlog is a living document: tasks close only through verified evidence, and the ledger of what closed, what failed, and what was deferred is this run's honest scorecard.

Risk register for the roadmap

Risk	Likelihood	Mitigation
Startup reorder exposes latent init-order assumptions	medium	boot with empty user-data in CI; stage guards; clean-start matrix
Store bounding loses data users care about	low	importance-aware eviction; migration reports trims; user-visible stores first
Deletion of legacy trio breaks unknown dependents	low	import-graph walk; test suite re-pointed; delete in one commit for revertability
Permission UI reveals mode semantics users dislike	medium	plain-language copy; defaults conservative; grants revocable
Windows-only paths regress unverified	medium	release checklist rows; launcher message box; Doctor probes

Table 31-2 · Roadmap risk register

How to know it worked

The roadmap's success test is behavioral, not bibliographic: a fresh machine goes from nothing to a companion on screen in minutes through the README's commands; a week of use leaves a store still under its budgets and a fast-path ratio still rising; a compound objective runs to a verified report with approvals where consequences live; and every failure the user meets comes with a reason they can act on. Those four observations, checked at each phase gate, are the difference between the vision being built

and merely being believed.

Phase gates and their artifacts

Gate	Must be true	Artifacts updated
P0 exit	startup budgets met; B1-B3, B6 verified closed	audit, bugs ledger, test report
P1 exit	single pipeline; bounded stores; permission UI live; digest real	audit, decisions, capability map
P2 exit	budgets measured in Doctor; observation dedupe live	test report, decisions
P3 exit	token scan at zero growth; six-companion pass; history ships	capability map, test report
P4 exit	procedures replay with trust scores; co-pilot observation behind grants	capability map, this document
P5 exit	checklists green; ledger at zero open high-severity	all artifacts synchronized

Table 31-3 · Phase gates with required artifact updates

The artifact column is not bureaucracy; it is how the next session starts from truth. Every gate closes with the repository's documents synchronized to the code's reality, and the discipline is the same one that governed this run: no claim without evidence, no closure without verification, no phase without its receipt.

32. Appendices

32. Appendices

Appendix A: Capability map legend

Status	Meaning
Live	Verified reachable from startup or IPC and exercised by tests or runtime
Partial	Present and wired but incomplete, gated, or missing UI
Planned	Priced in this document with a phase; not yet built
Blocked	Genuine external limitation documented (platform, license, or dependency)
Unverified	Code-reviewed but not executable in the audit sandbox (Windows-only paths)

Table 32-1 · Capability status legend used in CAPABILITY_MAP.md

Appendix B: IPC registry digest

The registry declares roughly sixty channels in one table shared by main and renderer, with the preload bridge exposing a flat, typed API object. The audit's contract checker validates registry, handler, and preload alignment, and the backlog extends it with renderer-usage coverage so dead channels fail CI. The digest's rule for the future: every channel is declared, typed, and used, or it is deleted, and new integrations, including any Skares-style capability, register here rather than alongside.

Appendix C: Glossary

Term	Definition
Understanding object	Structured output of the ten-stage pipeline: intents, entities, references, task shape, confidence
Expectation contract	A step's declared observable postcondition, checked by the verifier before success
Fast path	Deterministic resolution from index or procedure requiring zero model tokens
Procedure	Compact, parameterized, verified plan stored for replay; the unit of workflow learning
Permission mode	Safe / Standard / Dangerous-named setting governing which risk classes auto-run
Risk class	Single-source safety classification for every step, from the action contracts
Diff-only rescan	Recomputing environment state as changes against hashes, never as full rebuilds
Honest failure	A report naming step, expectation, observation, and attempted recovery; the only allowed failure format

Table 32-2 · Glossary of load-bearing terms

Appendix D: Document metadata

This document is the master vision and engineering blueprint for Quip, generated during the overnight engineering run at commit ae6b1ab and maintained as the source of truth alongside WORKLOG.md, BUGS_AND_FIXES.md, CODEBASE_AUDIT.md, ARCHITECTURE_DECISIONS.md, CAPABILITY_MAP.md, and FINAL_TEST_REPORT.md in the repository's docs directory. It supersedes scattered feature notes as the reference for direction, and it is versioned with the repository so the blueprint and the code disagree only in public, traceable ways.

Artifact index for this run

Artifact	Location	Purpose
Master Future Vision (this document)	docs/MASTER-FUTURE-VISION.pdf	the source of truth for direction
Engineering backlog (400+ tasks)	docs/BACKLOG.md	dependency-ordered execution ledger
Worklog	WORKLOG.md	chronological record of the run
Bugs and fixes	docs/BUGS_AND_FIXES.md	issue lifecycle from discovery to verified
Codebase audit	docs/CODEBASE_AUDIT.md	evidence-based findings with severity
Architecture decisions	docs/ARCHITECTURE_DECISIONS.md	the decisions and their reasons
Capability map	docs/CAPABILITY_MAP.md	live, partial, planned, blocked, unverified
Final test report	docs/FINAL_TEST_REPORT.md	what was actually tested and what it showed

Table 32-3 · The run's artifact index

A closing note on ownership

This document was written to be used, not admired. Its chapters are the reference for code reviews, its tables are the contracts new work must satisfy, and its audit is the receipt for what this run changed. When a future feature request arrives, the fastest path is to find its chapter, price it against the efficiency theses, and give it a backlog entry with a test. When something breaks, the audit's method, evidence first, root cause, fix, verification, is the template. Quip's promise to its user is honesty and capability in equal measure; this document's promise to its builders is the same thing in engineering form.

Reading the artifacts together

The seven artifacts divide the labor of truth: this document owns direction, the audit owns evidence, the decisions ledger owns rationale, the bugs ledger owns issue lifecycles, the capability map owns status, the backlog owns work, and the test report owns verification. They reference each other by name and none of them claims another's authority, which is what makes the set navigable after the run's context has faded. A future contributor holding only the repository can reconstruct this run's story from the artifacts in under an hour, and that reconstruction is the standard the artifacts set for themselves.

Document maintenance rules

The document changes under three triggers: architecture changes, decisions recorded in the decisions ledger update their chapter; capability changes, the registry and map updates update Chapters 5 through

9's status; and budget changes, any number in the tables moves with its measurement. It does not churn with features, and it does not carry dates it cannot defend. Versioning rides the repository, and the commit message rule is that document updates name their trigger, which keeps the history of the vision as legible as the vision itself.

The blueprint is done when the code no longer needs it and the team still reads it. Until then, it is the source of truth. After, it is the receipt.